PROMETEO · by thePower

SAD · SEGURIDAD Y ALTA DISPONIBILIDAD · ASIR 2.º

Práctica 1

Integridad, cifrado y firma con OpenSSL

Cuaderno de respuestas del alumno

Datos del alumno

Dato	Escribe aquí
Nombre y apellidos	Pedro Hosannah
Grupo	2º ASIR
Fecha de entrega	14/10/2026
Sistema operativo y terminal que usas (Git Bash, Terminal de Linux o macOS)	Ubuntu Linux (máquina virtual) en Paso 0, E1 y E2; Kali GNU/Linux Rolling 2026.2 (máquina virtual) desde E3. Terminal local y sesión SSH.
Versión de OpenSSL (salida de openssl version)	Ubuntu: OpenSSL 3.5.2 27 Jan 2026 (Library: OpenSSL 3.5.2 27 Jan 2026). Kali: OpenSSL 3.6.2 7 Apr 2026 (Library: OpenSSL 3.6.2 7 Apr 2026).

Cómo usar este cuaderno

1. Antes de empezar, guarda una copia de este archivo con el nombre Practica1_SAD_Apellido_Nombre, por ejemplo Practica1_SAD_Garcia_Lucia.docx. No uses acentos ni espacios en el nombre del archivo.
2. Rellena los datos de arriba. Escribe tus respuestas dentro de las casillas, bajo cada título. No borres los títulos ni cambies su orden, para que se pueda corregir.
3. Sigue el guión de la práctica paso a paso. Cada sección lleva el nombre del experimento (E1 a E5) y remite a los pasos del guión, por ejemplo el paso 2.4.
4. Para cada ejercicio entrega tres cosas. La orden que escribiste, la salida que obtuviste (texto o captura) y una frase razonada que explique qué se observa y por qué.
5. Para una captura en Windows pulsa Windows + Mayús + S, selecciona la zona y pégala en la casilla con Ctrl + V. Si copias texto de la terminal, pégalo con la fuente Consolas para que no se descoloque.
6. Trabaja siempre en tu propia máquina o en una máquina virtual de pruebas, y nunca uses contraseñas ni claves reales. Si una orden te pide una contraseña de prueba, usa una inventada.
7. Al terminar todo, crea un archivo zip con los ficheros que has generado en la práctica, como se explica en la sección final Entrega final, el archivo zip. En la entrega debes adjuntar los dos archivos, este cuaderno (en Word o en PDF) y el zip, dentro del plazo que indique el profesor.

Qué es una interpretación crítica

Copiar la salida de la terminal no es una respuesta. Lo que se valora es que la expliques y la cuestiones, en unas 4 a 8 frases. Para cada resultado, responde a estas cuatro preguntas con tus palabras y con lo que dice el material de la UT1.

- ¿Qué significa lo que ha salido? Explica cada dato y relaciónalo con el concepto de la UT1.
- ¿Coincide con lo que esperabas? Compáralo con tu predicción y explica la diferencia, si la hay.
- ¿Qué te sorprende, parece raro o podría ser un error? Duda de lo que ves y comprueba si tiene sentido.

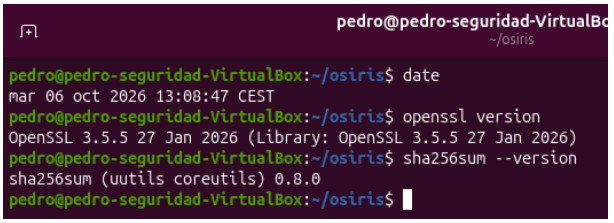
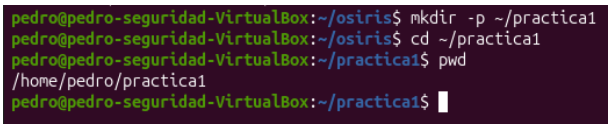
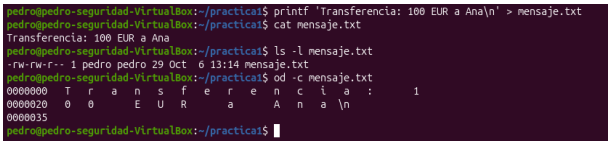
- ¿Qué límites tiene lo que has comprobado? Por ejemplo, un dato que depende de internet, de la fecha o de tu equipo.

Cómo se reparte el trabajo

Parte	Qué contiene	Se entrega en este cuaderno
Experimentos E1 a E5	Se hacen y se corrigen en clase, siguiendo el guión.	Sí, como registro de lo que hiciste
Ejercicios Ej1 a Ej6	Repiten lo aprendido con datos distintos y cada vez más difíciles.	Sí, es lo que se evalúa
Desafíos D1 a D4	Opcionales, se hacen fuera de clase.	Solo si los haces, indicando cuáles

Paso 0. Preparar el entorno

Pasos 0.1 a 0.3 del guión. Anota las órdenes y comprueba que todo responde antes de seguir.

Dato	Tu respuesta
Terminal que has usado	Terminal de Ubuntu Linux en la máquina virtual.
Primera línea de la salida de openssl version	OpenSSL 3.5.2 27 Jan 2026 (Library: OpenSSL 3.5.2 27 Jan 2026) **Comprobación de OpenSSL y sha256sum.** 
Primera línea de la salida de sha256sum --version	sha256sum (uutils coreutils) 0.8.0
Ruta que muestra la orden pwd	/home/pedro/practica1 **Creación de la carpeta de trabajo y comprobación con pwd.** 
Tamaño en bytes de mensaje.txt (orden ls -l)	29 bytes **Contenido final correcto de mensaje.txt.** 
¿Qué muestra od -c al final del fichero?	Al final aparece \n, es decir, el salto de línea final que no se ve al mostrar el texto normalmente.

Si algo no salió como en la guía
Cuenta qué mensaje apareció (por ejemplo command not found), qué revisaste y cómo lo resolviste. Si todo fue bien, escribe "sin incidencias".

Al principio escribí "ANA" en mayúsculas en lugar de "Ana". El fichero seguía ocupando 29 bytes, pero el contenido no era exactamente el del guion y habría producido una huella SHA-256 distinta. Lo corregí antes de empezar E1 y volví a comprobar el contenido con cat, ls -l y od -c.

¿Por qué importan los 29 bytes de mensaje.txt?
Piensa en el salto de línea que no se ve y en lo que hará el hash con él.

Importan porque SHA-256 tiene en cuenta todos los bytes del fichero, incluido el salto de línea que no se ve al mostrar el texto. mensaje.txt tiene 28 caracteres visibles y el \n final, que suma otro byte. Si ese byte cambia o desaparece, la huella también cambia.

E1. Hash e integridad

Pasos 1.1 a 1.6 del guión. Anota las órdenes que escribiste y pega lo que salió en pantalla.

Pasos 1.1 a 1.3, la huella y el efecto avalancha

Dato	Tu respuesta
Huella SHA-256 de mensaje.txt (primeros 8 caracteres)	2fea0329
¿Cuántos caracteres tiene la huella?	64 caracteres
Huella de mensaje2.txt (primeros 8 caracteres)	62d632ef
Resultado de distintos mensaje.txt mensaje2.txt	63 de 64 caracteres
Resultado de distintos mensaje.txt mensaje3.txt	63 de 64 caracteres
Resultado de distintos mensaje.txt mensaje4.txt	62 de 64 caracteres

Antes de ejecutar, escribe tu apuesta	Tu respuesta
Predicción 1. Voy a cambiar el importe de 100 a 900. De los 64 caracteres de la huella, ¿cambiarán pocos, la mitad o casi todos?	Creo que cambiarán casi todos los caracteres de la huella, aunque solo cambie un carácter del contenido.
Lo que ocurrió	Cambió prácticamente toda la huella: la comparación dio 63 caracteres distintos de 64.
¿Acertaste? ¿Qué no habías visto?	Sí, acerté. Lo que comprobé es que un cambio mínimo en el fichero provoca un cambio muy grande en la huella, que es el efecto avalancha.

Paso 1.4, la huella siempre tiene el mismo tamaño

Fichero	Tamaño en bytes	Longitud de su huella	Huella (primeros 8 caracteres)
mensaje.txt	29	64	2fea0329
grande.bin	10485760	64	cb4b813b
vacio.txt	0	64	e3b0c442

Pasos 1.5 y 1.6, detectar la manipulación

Situación	Resultado de sha256sum -c	¿Qué significa?
Fichero sin tocar (paso 1.5)	mensaje.txt: OK	La huella actual coincide con la guardada en mensaje.sha256; el fichero no ha cambiado.
Después de añadir una línea con echo "x" >>	mensaje.txt: FAILED	La línea añadida con echo modificó el contenido y la huella dejó de coincidir.
Después de restaurar el original	mensaje.txt: OK	Al restaurar exactamente el contenido original, la huella volvió a coincidir.
Carpeta: copia/f2.txt modificado	copia/f2.txt: FAILED	El fichero sigue existiendo, pero su contenido fue modificado.

Situación	Resultado de sha256sum -c	¿Qué significa?
Carpeta: copia/f4.txt borrado	copia/f4.txt: FAILED open or read / No such file or directory	El manifiesto esperaba ese fichero, pero fue borrado y ya no puede leerlo.
Carpeta: copia/nuevo.txt añadido	No aparece en la comprobación	nuevo.txt se añadió después de crear el manifiesto, por eso sha256sum -c no lo conoce y no lo detecta.

Qué escribí

Copia las órdenes de los pasos 1.1 a 1.6, incluida la función distintos.

```
sha256sum mensaje.txt
printf 'Transferencia: 900 EUR a Ana\n' > mensaje2.txt
sha256sum mensaje2.txt
distintos() { a=$(sha256sum "$1" | cut -d" " -f1); b=$(sha256sum "$2" | cut -d" " -f1); n=0; for i in $(seq 0 63);
do [ "${a:$i:1}" != "${b:$i:1}" ] && n=$((n+1)); done; echo "$1 y $2 difieren en $n de 64 caracteres"; }
distintos mensaje.txt mensaje2.txt
printf 'Transferencia: 101 EUR a Ana\n' > mensaje3.txt
printf 'Transferencia: 100 EUR a Ana \n' > mensaje4.txt
distintos mensaje.txt mensaje3.txt
distintos mensaje.txt mensaje4.txt
head -c 10485760 /dev/urandom > grande.bin
ls -l grande.bin
sha256sum grande.bin
touch vacio.txt
sha256sum vacio.txt
sha256sum mensaje.txt > mensaje.sha256
cat mensaje.sha256
sha256sum -c mensaje.sha256
echo "x" >> mensaje.txt
sha256sum -c mensaje.sha256
printf 'Transferencia: 100 EUR a Ana\n' > mensaje.txt
sha256sum -c mensaje.sha256
mkdir copia
for i in 1 2 3 4 5; do printf "contenido del fichero $i\n" > copia/f$i.txt; done
sha256sum copia/* > manifiesto.sha256
sha256sum -c manifiesto.sha256
echo "extra" >> copia/f2.txt
rm copia/f4.txt
printf "nuevo\n" > copia/nuevo.txt
sha256sum -c manifiesto.sha256
```

Lo que apareció en pantalla

Pega la salida o una captura. En el paso 1.6 debe verse el resultado de las cuatro comprobaciones.

Paso 1.1. Huella SHA-256 de mensaje.txt.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum mensaje.txt
2fea0329539adcd45137d6dea80dda0d3e3e5459e28b91935da87cb9faec23c1  mensaje.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 1.2. Huella de mensaje2.txt tras cambiar 100 por 900.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ printf 'Transferencia: 900 EUR a Ana\n' > mensaje2.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum mensaje2.txt
62d632efb08e9e86c7927093ac92f1185871cf4e243858359bf3a3763cdada39  mensaje2.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 1.3. Función distintos y comparación inicial: 63 de 64 caracteres.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ distintos() { a=$(sha256sum "$1" | cut -d" " -f1); b=$(sha256su
m "$2" | cut -d" " -f1); n=0; for i in $(seq 0 63); do [ "${a:$i:1}" != "${b:$i:1}" ] && n=$((n+1)); done; ec
ho "$1 y $2 difieren en $n de 64 caracteres"; }
pedro@pedro-seguridad-VirtualBox:~/practica1$ distintos mensaje.txt mensaje2.txt
mensaje.txt y mensaje2.txt difieren en 63 de 64 caracteres
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 1.3. Cambios pequeños e invisible: 63/64 y 62/64 caracteres distintos.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ printf 'Transferencia: 101 EUR a Ana\n' > mensaje3.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ printf 'Transferencia: 100 EUR a Ana \n' > mensaje4.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ distintos mensaje.txt mensaje3.txt
mensaje.txt y mensaje3.txt difieren en 63 de 64 caracteres
pedro@pedro-seguridad-VirtualBox:~/practica1$ distintos mensaje.txt mensaje4.txt
mensaje.txt y mensaje4.txt difieren en 62 de 64 caracteres
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Activar Windows
Ve a Configuración para activar Windows.

Paso 1.4. Fichero de 10 MiB y fichero vacío: ambas huellas tienen longitud fija.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ head -c 10485760 /dev/urandom > grande.bin
pedro@pedro-seguridad-VirtualBox:~/practica1$ ls -l grande.bin
-rw-rw-r-- 1 pedro pedro 10485760 Oct  6 13:25 grande.bin
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum grande.bin
cb4b813bf9315b9e6ee39061f063e59f4f5af73c125b119db15d153f75da7a47  grande.bin
pedro@pedro-seguridad-VirtualBox:~/practica1$ touch vacio.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum vacio.txt
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  vacio.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Activar Windows
Ve a Configuración para activar Windows.

Paso 1.5. Comprobación OK, detección de la modificación y restauración.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum mensaje.txt > mensaje.sha256
pedro@pedro-seguridad-VirtualBox:~/practica1$ cat mensaje.sha256
2fea0329539adcd45137d6dea80dda0d3e3e5459e28b91935da87cb9faec23c1  mensaje.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum -c mensaje.sha256
mensaje.txt: OK
pedro@pedro-seguridad-VirtualBox:~/practica1$ echo "x" >> mensaje.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum -c mensaje.sha256
mensaje.txt: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
pedro@pedro-seguridad-VirtualBox:~/practica1$ printf 'Transferencia: 100 EUR a Ana\n' > mensaje.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum -c mensaje.sha256
mensaje.txt: OK
çpedro@pedro-seguridad-VirtualBox:~/practica1$
```

Activar Windows
Ve a Configuración para activar Windows.

Paso 1.6. Manifiesto de una carpeta: fichero modificado, borrado y fichero nuevo no registrado.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ mkdir copia
pedro@pedro-seguridad-VirtualBox:~/practica1$ for i in 1 2 3 4 5; do printf "contenido del fichero $i\n" > co
pia/f$i.txt; done
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum copia/* > manifiesto.sha256
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum -c manifiesto.sha256
copia/f1.txt: OK
copia/f2.txt: OK
copia/f3.txt: OK
copia/f4.txt: OK
copia/f5.txt: OK
pedro@pedro-seguridad-VirtualBox:~/practica1$ echo "extra" >> copia/f2.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ rm copia/f4.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ printf "nuevo\n" > copia/nuevo.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum -c manifiesto.sha256
copia/f1.txt: OK
copia/f2.txt: FAILED
copia/f3.txt: OK
sha256sum: copia/f4.txt: No such file or directory
copia/f4.txt: FAILED open or read
copia/f5.txt: OK
sha256sum: WARNING: 1 computed checksum did NOT match
sha256sum: WARNING: 1 listed file could not be read
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Interpretación y análisis crítico

Explica por qué casi toda la huella cambia con un solo carácter (efecto avalancha) y por qué dos huellas aleatorias coinciden en cerca de 1 de cada 16 caracteres. Compara con tu predicción. ¿Se detectó nuevo.txt? ¿Qué limitación tiene el manifiesto? ¿Qué pasaría si el atacante también modifica mensaje.sha256?

Al cambiar un solo carácter de mensaje.txt, la huella cambió casi por completo: obtuve 63 diferencias de 64. Ese cambio tan grande es el efecto avalancha. Como cada posición hexadecimal puede tener 16 valores, dos huellas distintas pueden coincidir por casualidad en aproximadamente 1 de cada 16 posiciones; por eso es normal que la mayoría cambie. En mis pruebas salieron 63, 63 y 62 diferencias, incluso al añadir solo un espacio antes del salto de línea. El manifiesto detectó f2.txt modificado y f4.txt borrado, pero no nuevo.txt porque se creó después. Si un atacante pudiera modificar también mensaje.sha256 o el manifiesto, podría recalcular las huellas y ocultar el cambio, así que esa referencia también debe protegerse o firmarse.

Respuesta a la pregunta del paso 1.4

¿Podrías reconstruir los 10 MiB a partir de 64 caracteres? ¿Por qué no?

No. SHA-256 transforma datos de cualquier tamaño en una salida fija de 256 bits, representada con 64 caracteres hexadecimales. Al reducir un fichero de 10 MiB a una huella tan pequeña se pierde información, y la función está diseñada para ser unidireccional, por lo que no permite reconstruir el contenido original a partir de la huella.

Conclusión del experimento

Entre 2 y 4 frases con la idea principal que te llevas.

En este experimento vi que SHA-256 siempre genera una huella de longitud fija y que un cambio mínimo modifica casi toda la huella. El manifiesto me sirvió para detectar ficheros modificados o borrados que ya estaban registrados, pero no archivos nuevos añadidos después. También vi que la huella o el manifiesto tienen que estar protegidos para que la comprobación sea fiable.

Preguntas de comprensión

Pregunta	Tu respuesta
¿Cuántos caracteres tiene una huella SHA-256 y por qué no depende del tamaño del fichero?	Una huella SHA-256 tiene 64 caracteres hexadecimales porque representa 256 bits. Su longitud la fija el algoritmo, por eso es la misma tanto para un fichero vacío como para uno de 10 MiB.

Pregunta	Tu respuesta
Si dos ficheros tienen la misma huella, ¿qué puedes concluir? ¿Y si tienen huellas distintas?	Si dos ficheros tienen la misma huella SHA-256, en la práctica tengo una evidencia muy fuerte de que contienen los mismos datos, aunque teóricamente pueden existir colisiones. Si las huellas son distintas, los ficheros no son idénticos.
¿Por qué no basta con guardar la huella en el mismo sitio que el fichero que protege?	Porque si un atacante puede modificar el fichero protegido y también la huella guardada a su lado, puede recalcularla y hacer que la comprobación vuelva a dar OK. La referencia debe guardarse en un lugar protegido o estar firmada.

E2. Cifrado simétrico con AES-256

Pasos 2.1 a 2.5 del guión. En el cuaderno usa solo contraseñas de prueba.

Dato	Tu respuesta
Contraseña de prueba que usaste (inventada)	qwerty12345 (contraseña de prueba)
Tamaño en bytes de mensaje.txt	29 bytes
Tamaño en bytes de mensaje.enc	48 bytes
Qué letras aparecen en la primera línea de od -c mensaje.enc	Salted__
Resultado de cat mensaje.dec	Transferencia: 100 EUR a Ana
Mensaje y código de salida con la contraseña incorrecta (echo $?)	bad decrypt; código de salida 1
¿Coinciden las huellas de copia1.enc y copia2.enc?	No. Las huellas de copia1.enc y copia2.enc son distintas.
¿Se descifran los dos ficheros igual?	Sí. Los dos ficheros se descifran como: Transferencia: 100 EUR a Ana.

Antes de ejecutar, escribe tu apuesta	Tu respuesta
Predicción 2. Voy a descifrar con una contraseña distinta. ¿Obtendré un texto sin sentido o un error?	Creo que obtendré un error al usar una contraseña incorrecta, no el texto original.
Lo que ocurrió	OpenSSL mostró "bad decrypt" y el código de salida fue 1.
¿Acertaste? ¿Qué no habías visto?	Sí, acerté con que aparecería un error. Lo que no había tenido en cuenta es que bad decrypt suele aparecer porque el relleno final no es válido y que AES-CBC, por sí solo, no comprueba la integridad del fichero.

Qué escribí
Copia las órdenes de los pasos 2.1 a 2.5.

```
openssl enc -aes-256-cbc -pbkdf2 -salt -in mensaje.txt -out mensaje.enc
ls -l mensaje.txt mensaje.enc
od -c mensaje.enc | head -4
openssl enc -d -aes-256-cbc -pbkdf2 -in mensaje.enc -out mensaje.dec
cat mensaje.dec
diff mensaje.txt mensaje.dec && echo "Descifrado correcto"
openssl enc -d -aes-256-cbc -pbkdf2 -in mensaje.enc -out mensaje.mal
echo $?
openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:qwerty12345 -in mensaje.txt -out copia1.enc
openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:qwerty12345 -in mensaje.txt -out copia2.enc
sha256sum copia1.enc copia2.enc
openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:qwerty12345 -in copia1.enc
openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:qwerty12345 -in copia2.enc
```

Lo que apareció en pantalla
Pega una captura o copia el texto de la salida de cada orden. Si una orden dio error, pégalo también.

Paso 2.1. Cifrado de mensaje.txt con AES-256-CBC.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ openssl enc -aes-256-cbc -pbkdf2 -salt -in mensaje.txt -out mensaje.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 2.2. Comparación de tamaños y cabecera Salted__ del fichero cifrado.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ ls -l mensaje.txt mensaje.enc
-rw-rw-r-- 1 pedro pedro 48 Oct  7 13:46 mensaje.enc
-rw-rw-r-- 1 pedro pedro 29 Oct  6 13:27 mensaje.txt
pedro@pedro-seguridad-VirtualBox:~/practica1$ od -c mensaje.enc | head -4
0000000   S   a   l   t   e   d   _   _   (  250   N   t 211 273   p 330
0000020 030   w 300   # 326 210   R   ;   u 247 277 335 301   C   . 312
0000040 216 367   - 323 202 322 331   6 324 373   / 343 317 230 216   q
0000060
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 2.3. Descifrado con la contraseña correcta y comprobación con diff.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ openssl enc -d -aes-256-cbc -pbkdf2 -in mensaje.enc -out mensaje.dec
enter AES-256-CBC decryption password:
pedro@pedro-seguridad-VirtualBox:~/practica1$ cat mensaje.dec
Transferencia: 100 EUR a Ana
pedro@pedro-seguridad-VirtualBox:~/practica1$ diff mensaje.txt mensaje.dec && echo "Descifrado correcto"
Descifrado correcto
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 2.4. Contraseña incorrecta: bad decrypt y código de salida 1.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ openssl enc -d -aes-256-cbc -pbkdf2 -in mensaje.enc -out mensaje.mal
enter AES-256-CBC decryption password:
bad decrypt
4007C44E18780000:error:1C800064:Provider routines:ossl_cipher_unpadblock:bad decrypt:../providers/implementations/ciphers/ciphercomm
on_block.c:107:
pedro@pedro-seguridad-VirtualBox:~/practica1$ echo $?
1
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 2.5. Dos cifrados del mismo mensaje: las huellas son diferentes por la sal.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:qwerty12345 -in mensaje.txt -out cop
ia1.enc
pedro@pedro-seguridad-VirtualBox:~/practica1$ openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:qwerty12345 -in mensaje.txt -out cop
ia2.enc
pedro@pedro-seguridad-VirtualBox:~/practica1$ sha256sum copia1.enc copia2.enc
906d68514a011c95cacc4afb7d04560b00ad86833874b17dc1ce3924f8234256  copia1.enc
d756e56d487c9ac1197b3b884b84e1052505838822ba0f3d427b6b7317c3f67c  copia2.enc
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Paso 2.5. Ambos ficheros cifrados recuperan exactamente el mismo mensaje.

```
pedro@pedro-seguridad-VirtualBox:~/practica1$ openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:qwerty12345 -in copia1.enc
Transferencia: 100 EUR a Ana
pedro@pedro-seguridad-VirtualBox:~/practica1$ openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:qwerty12345 -in copia2.enc
Transferencia: 100 EUR a Ana
pedro@pedro-seguridad-VirtualBox:~/practica1$
```

Por qué ha crecido el fichero (29 a 48 bytes)

Descompón los 48 bytes con lo que explica el guión sobre la cabecera, la sal y el relleno.

mensaje.txt ocupa 29 bytes y mensaje.enc ocupa 48 bytes. Los primeros 16 bytes del fichero cifrado corresponden a la cabecera: 8 bytes con "Salted__" y 8 bytes de sal. Los otros 32 bytes son los datos cifrados. Como AES trabaja con bloques de 16 bytes, los 29 bytes del mensaje necesitan relleno hasta completar 32 bytes antes de cifrarse. Por eso el total final es 16 + 32 = 48 bytes.

Interpretación y análisis crítico

Explica qué contiene mensaje.enc y por qué no deja ver el texto original. ¿Por qué falló el descifrado con bad decrypt y por qué eso no prueba que nadie haya manipulado el fichero? ¿Qué ventaja da la sal frente a quien observe muchos ficheros cifrados?

En mensaje.enc ya no se puede leer la frase original porque el fichero contiene la cabecera Salted__, la sal y los datos cifrados. Con la contraseña correcta recuperé el mensaje y diff indicó que el fichero descifrado era idéntico al original. Al usar una contraseña incorrecta, OpenSSL mostró bad decrypt y código 1, normalmente porque el relleno final no era válido. Ese error no demuestra por sí solo que el fichero haya sido manipulado, ya que AES-CBC no autentica el contenido. También cifré dos veces el mismo mensaje con qwerty12345: las huellas fueron distintas por la sal aleatoria, aunque ambos ficheros se descifraron al mismo texto. Así no se puede reconocer a simple vista que parten del mismo contenido y contraseña. En el primer intento del paso 2.4 puse por error la contraseña correcta; borré el fichero generado y repetí la prueba con una contraseña distinta.

Conclusión del experimento
Entre 2 y 4 frases con la idea principal que te llevas.

Con las pruebas de AES-256-CBC vi que puedo cifrar y recuperar un fichero usando la misma contraseña, protegiendo su confidencialidad. La sal hace que dos cifrados del mismo contenido salgan diferentes. También comprobé que un error al descifrar no sustituye a un mecanismo de integridad o autenticación.

Preguntas de comprensión

Pregunta	Tu respuesta
¿Por qué no se puede enviar la contraseña en el mismo correo que el fichero cifrado?	Porque si alguien intercepta el mismo correo tendría a la vez el fichero cifrado y la contraseña, y podría descifrarlo. La contraseña debe enviarse por un canal separado y seguro.
¿Para qué sirve la sal y dónde se guarda?	La sal añade un valor aleatorio al proceso de derivación de la clave para que el mismo contenido y contraseña no produzcan siempre el mismo cifrado. Se guarda dentro del propio fichero cifrado, junto a la cabecera Salted__, y no necesita ser secreta.
Si alguien cambia unos bytes del fichero cifrado, ¿lo detectarías? Responde con lo que viste en el paso 2.4.	No de forma fiable. En la prueba apareció bad decrypt porque el relleno final no era válido, pero AES-CBC por sí solo no autentica el contenido. Una modificación podría no detectarse siempre; para comprobar la integridad haría falta una firma o un modo autenticado, como AES-GCM.

E3. Criptografía asimétrica y firma digital

Pasos 3.1 a 3.6 del guión. La clave privada no sale nunca de tu equipo ni se pega en este cuaderno. Pega solo las primeras líneas, nunca el contenido completo.

Dato	Tu respuesta
Primera línea de privada.pem	-----BEGIN PRIVATE KEY-----
Primera línea de publica.pem	-----BEGIN PUBLIC KEY-----
Salida de openssl pkey -text -noout | head -1	Private-Key: (3072 bit, 2 primes)
Tamaño de mensaje.sig en bytes	384 bytes
Tamaño de la firma de grande.bin en bytes	384 bytes
Resultado de verificar la firma original	Verified OK
Resultado de verificar tras añadir un espacio	Verification failure (bad signature)
Resultado de verificar tras restaurar el mensaje	Verified OK
Tamaño de secreto.enc en bytes	384 bytes
Texto que devuelve el descifrado con la clave correcta	La clave del servidor es 1234
Error al descifrar con privada2.pem	Public Key operation error; data greater than mod len
Error al cifrar largo.txt con RSA	Public Key operation error; data too large for key size

Antes de ejecutar, escribe tu apuesta	Tu respuesta
Predicción 3. Voy a añadir un espacio al final del mensaje y a verificar la misma firma. ¿Verificará?	Creo que no verificará, porque la firma está asociada a los bytes exactos del mensaje y al modificarlo la huella ya no será la misma.
Lo que ocurrió	La verificación falló y OpenSSL mostró "Verification failure" y un error de "bad signature". Al restaurar el mensaje original volvió a aparecer "Verified OK".
¿Acertaste? ¿Qué no habías visto?	Sí, acerté. Lo que comprobé es que incluso un cambio mínimo invalida la firma. Al restaurar exactamente el fichero original, la misma firma volvió a ser válida.

Qué escribí
Copia las órdenes de los pasos 3.1 a 3.6.

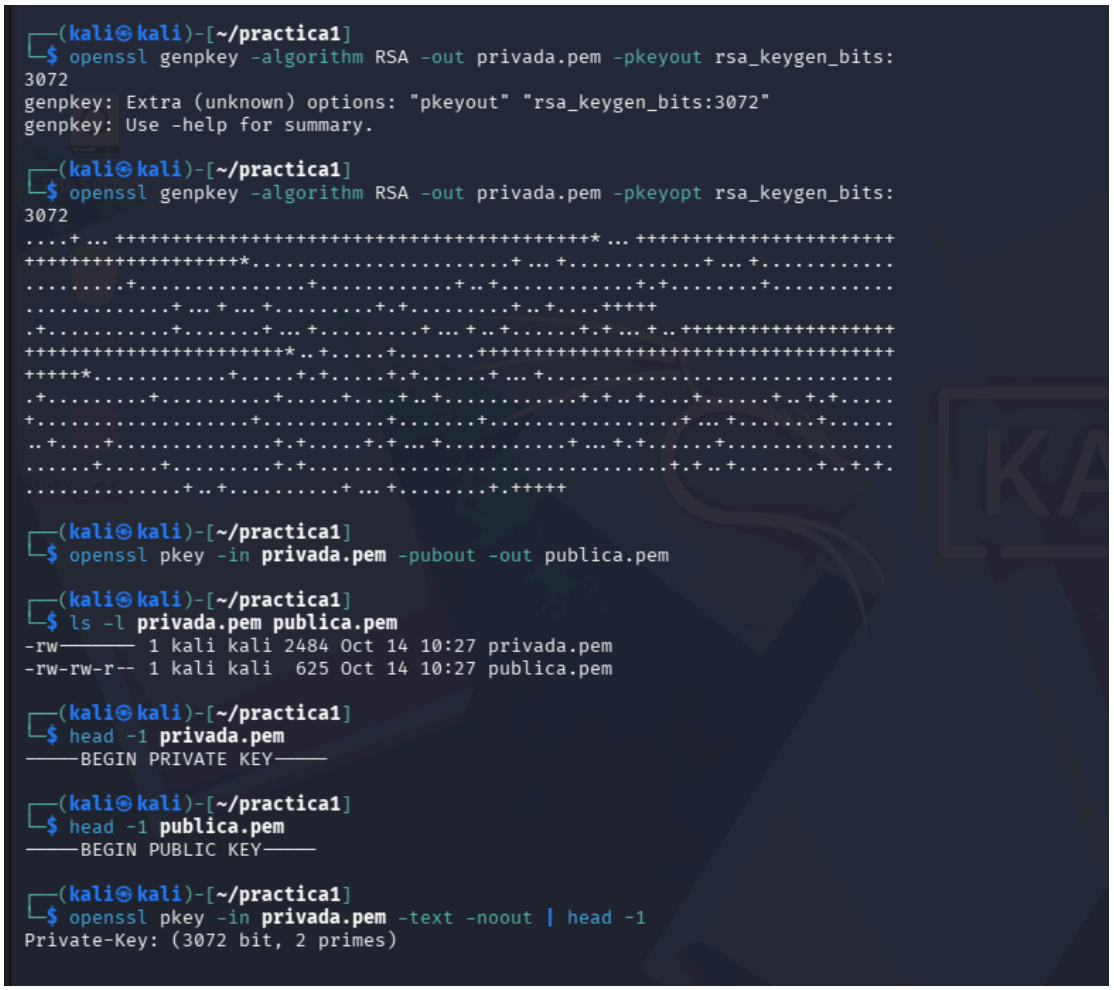
Incidencia al retomar la práctica: la máquina virtual Ubuntu utilizada para Paso 0, E1 y E2 dejó de arrancar correctamente y los archivos de trabajo ya no estaban disponibles. Para poder continuar, preparé una nueva máquina virtual con Kali GNU/Linux Rolling 2026.2 y reconstruí únicamente los ficheros necesarios para seguir desde E3. Los apartados anteriores no se volvieron a documentar porque ya estaban realizados y registrados.

```
openssl genpkey -algorithm RSA -out privada.pem -pkeyopt rsa_keygen_bits:3072
openssl pkey -in privada.pem -pubout -out publica.pem
ls -l privada.pem publica.pem
head -1 privada.pem
head -1 publica.pem
openssl pkey -in privada.pem -text -noout | head -1
sha256sum mensaje.txt
openssl dgst -sha256 -sign privada.pem -out mensaje.sig mensaje.txt
ls -l mensaje.sig
openssl dgst -sha256 -sign privada.pem -out grande.sig grande.bin
ls -l grande.sig
openssl dgst -sha256 -verify publica.pem -signature mensaje.sig mensaje.txt
echo " " >> mensaje.txt
openssl dgst -sha256 -verify publica.pem -signature mensaje.sig mensaje.txt
printf 'Transferencia: 100 EUR a Ana\n' > mensaje.txt
openssl dgst -sha256 -verify publica.pem -signature mensaje.sig mensaje.txt
printf 'La clave del servidor es 1234\n' > secreto.txt
openssl pkeyutl -encrypt -pubin -inkey publica.pem -in secreto.txt -out secreto.enc
ls -l secreto.enc
openssl pkeyutl -decrypt -inkey privada.pem -in secreto.enc
openssl genpkey -algorithm RSA -out privada2.pem -pkeyopt rsa_keygen_bits:2048
openssl pkeyutl -decrypt -inkey privada2.pem -in secreto.enc
head -c 400 /dev/zero | tr '\0' 'a' > largo.txt
openssl pkeyutl -encrypt -pubin -inkey publica.pem -in largo.txt -out largo.enc
```

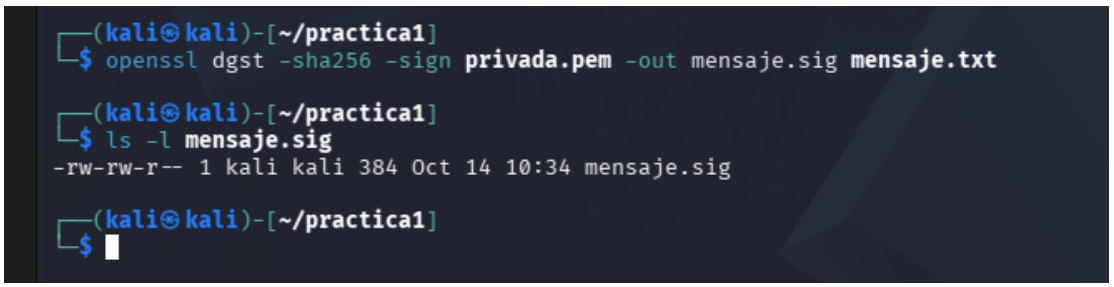
Lo que apareció en pantalla

Pega una captura o copia el texto de la salida de cada orden. Si una orden dio error, pégalo también.

Paso 3.1. Generación y comprobación del par de claves RSA. También se observa el error inicial al escribir -pkeyout en lugar de -pkeyopt.



Paso 3.2. Firma de mensaje.txt: mensaje.sig ocupa 384 bytes.



Paso 3.2. Firma de grande.bin: grande.sig también ocupa 384 bytes.



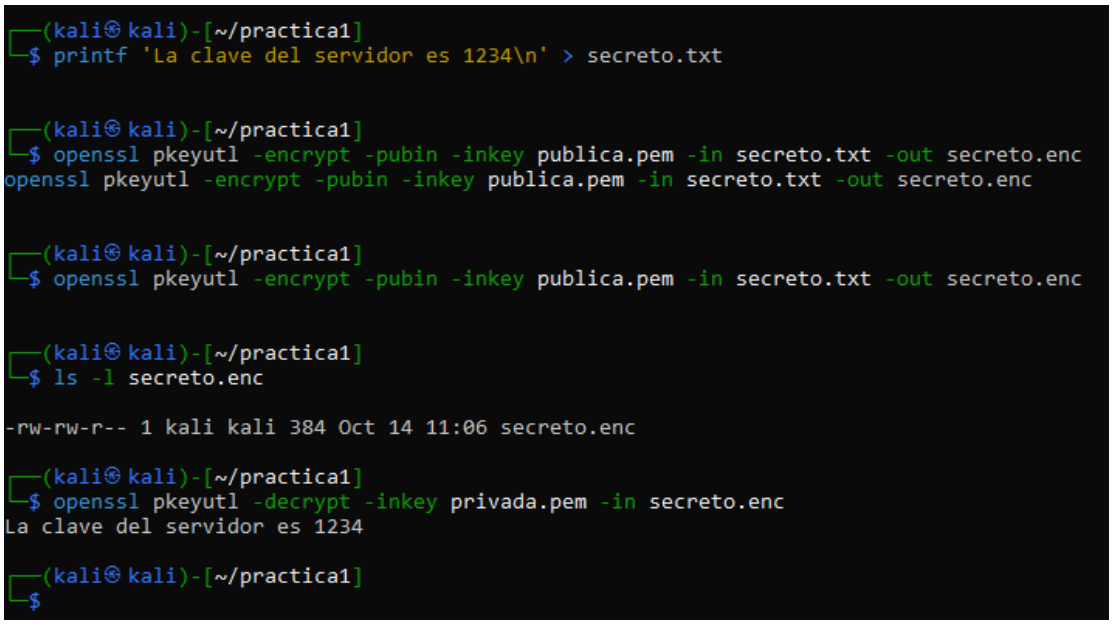
Paso 3.3. Verificación de la firma original con la clave pública.



Paso 3.4. La firma falla tras modificar el mensaje y vuelve a verificar al restaurarlo.



Paso 3.5. Cifrado con la clave pública y descifrado con la clave privada correspondiente.



Paso 3.6. Una clave privada distinta no descifra el secreto y RSA rechaza un dato demasiado grande.



Interpretación y análisis crítico

Explica qué se firma, con qué clave y qué garantiza la firma y qué no. ¿Por qué el tamaño de la firma no depende del tamaño del mensaje? ¿Por qué RSA no sirve para cifrar ficheros grandes y cómo se resuelve en la práctica?

Al firmar, OpenSSL no firma directamente todo el fichero, sino su huella SHA-256, usando la clave privada. La clave pública sirve para verificar que la firma corresponde al mensaje y que el contenido no ha cambiado desde que se firmó. En mis pruebas, mensaje.sig y la firma de grande.bin ocuparon 384 bytes aunque los ficheros tenían tamaños muy distintos, porque el tamaño de la firma depende de la clave RSA de 3072 bits. Al modificar mensaje.txt apareció Verification failure y, al restaurarlo, volvió a dar Verified OK. Para la prueba de confidencialidad cifré secreto.txt con la clave pública y lo recuperé con la privada. También vi que otra clave privada no podía descifrarlo y que RSA no aceptaba directamente un fichero de 400 bytes; por eso, para ficheros grandes, se usa normalmente cifrado simétrico y RSA se reserva para datos pequeños, como una clave AES.

Conclusión del experimento
Entre 2 y 4 frases con la idea principal que te llevas.

En E3 vi claramente la diferencia entre firmar y cifrar con criptografía asimétrica. La clave privada se usa para firmar y para descifrar lo destinado a su propietario, mientras que la pública se puede compartir para verificar firmas o cifrar información para ese destinatario. También comprobé que RSA no es adecuado para cifrar ficheros grandes directamente.

Preguntas de comprensión

Pregunta	Tu respuesta
¿Qué clave puedes enviar por correo sin riesgo y cuál no debe salir de tu equipo?	Puedo enviar la clave pública, porque está pensada para compartirse. La clave privada no debe salir de mi equipo ni entregarse a otra persona.
¿Con qué clave se firma, con cuál se verifica y con cuál se cifra para que solo lea el destinatario?	Se firma con la clave privada y se verifica con la pública. Para que solo el destinatario pueda leer un dato, se cifra con su clave pública y él lo descifra con su clave privada.
¿Qué garantiza una firma y qué no garantiza?	Una firma permite comprobar el origen de la firma y la integridad del contenido: si el mensaje cambia, deja de verificar. No aporta confidencialidad, porque el mensaje firmado puede seguir siendo legible si no se cifra.

E4. El certificado de un servidor real

Pasos 4.1 a 4.5 del guión. Este experimento necesita internet y los datos cambian con el tiempo, así que es normal que no coincidan con los de tus compañeros. Anota la fecha en que lo hiciste.

Dato	Tu respuesta
Fecha y hora en que hiciste el experimento	14/10/2026, aproximadamente 17:12 (hora local)
Titular del certificado (subject)	CN=*.wikipedia.org
Emisor (issuer)	C=US, O=Let's Encrypt, CN=E2
Inicio de validez (notBefore)	Oct 4 18:34:51 2026 GMT
Fin de validez (notAfter)	Jan 2 18:34:50 2027 GMT
Número de certificados que envía el servidor	4 certificados
Algunos nombres de la lista subjectAltName	Entre otros: *.wikipedia.org, wikipedia.org, *.wikimedia.org, wikimedia.org, *.wikibooks.org, wikibooks.org, *.wikidata.org y wikidata.org.
Resultado con el nombre correcto (Verify return code)	Verification: OK / Verify return code: 0 (ok)
Resultado con otro nombre (Verify return code)	Verification error: hostname mismatch / Verify return code: 62 (hostname mismatch)
Resultado con la fecha adelantada dos años	Verification error: certificate has expired / Verify return code: 10 (certificate has expired)

Antes de ejecutar, escribe tu apuesta	Tu respuesta
Predicción 4. Voy a pedir a OpenSSL que verifique el mismo certificado primero con el nombre correcto y después con un nombre que no es el de la web. ¿Qué respuesta espero en cada caso?	Con www.wikipedia.org espero una verificación correcta. Con otro.ejemplo.com espero un error porque ese nombre no está incluido entre los nombres válidos del certificado.
Lo que ocurrió	Con www.wikipedia.org apareció Verification: OK y código 0. Con otro.ejemplo.com apareció hostname mismatch y código 62.
¿Acertaste? ¿Qué no habías visto?	Sí, acerté. Además comprobé que OpenSSL distingue claramente un problema de nombre de un problema de fecha: al simular el reloj adelantado dos años apareció el código 10 por certificado caducado.

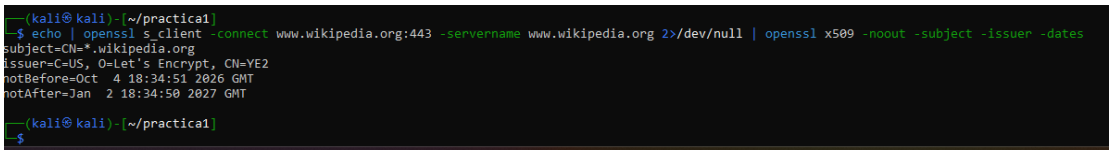
Qué escribí
Copia las órdenes de los pasos 4.1 a 4.5.

```
echo | openssl s_client -connect www.wikipedia.org:443 -servername www.wikipedia.org 2>/dev/null | openssl x509 -noout -subject -issuer -dates
echo | openssl s_client -connect www.wikipedia.org:443 -servername www.wikipedia.org -showcerts 2>/dev/null | grep -c "BEGIN CERTIFICATE"
echo | openssl s_client -connect www.wikipedia.org:443 -servername www.wikipedia.org 2>/dev/null | openssl x509 -noout -ext subjectAltName
echo | openssl s_client -connect www.wikipedia.org:443 -servername www.wikipedia.org -verify_hostname www.wikipedia.org 2>&1 | grep -E "Verification|Verify return code"
echo | openssl s_client -connect www.wikipedia.org:443 -servername www.wikipedia.org -verify_hostname otro.ejemplo.com 2>&1 | grep -E "Verification|Verify return code"
echo | openssl s_client -connect www.wikipedia.org:443 -servername www.wikipedia.org -attime $(( $(date +%s) + 63072000 )) 2>&1 | grep -E "Verification|Verify return code"
```

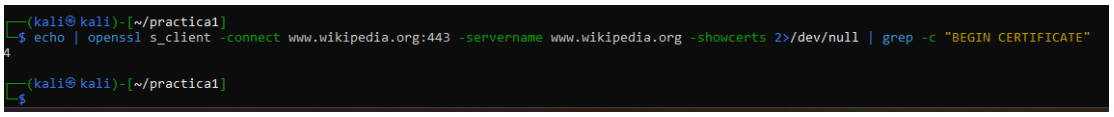
Lo que apareció en pantalla

Pega una captura o copia el texto de la salida de cada orden. Si una orden dio error, pégalo también.

Paso 4.1. Titular, emisor y periodo de validez del certificado de Wikipedia.



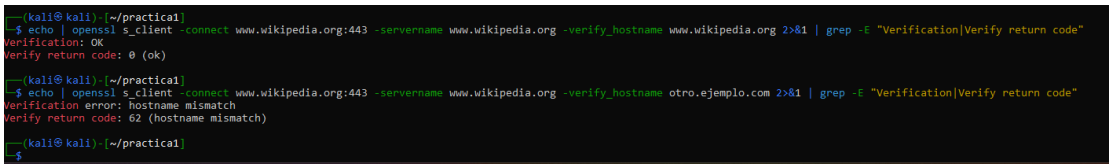
Paso 4.2. El servidor envió 4 certificados en la cadena durante la prueba.



Paso 4.3. Nombres incluidos en subjectAltName.



Paso 4.4. Verificación correcta con www.wikipedia.org y fallo de hostname con otro.ejemplo.com.



Paso 4.5. Simulación de un reloj adelantado dos años: certificado considerado caducado.



Interpretación y análisis crítico

Explica quién es el titular y quién lo emitió, y si el titular coincide con la web que pediste. ¿Por qué el servidor envía los intermedios y no la raíz? Si apareció unable to get local issuer certificate, cuéntalo y explica por qué no es un ataque. ¿Qué causa de aviso habitual ilustra cada una de las tres pruebas del paso 4.4 y 4.5?

En mi prueba, el certificado tenía como titular CN=*.wikipedia.org, que cubre www.wikipedia.org, y estaba emitido por Let's Encrypt, CN=E2. El servidor envió 4 certificados; ese número puede cambiar con el tiempo según la cadena que use. Los intermedios sirven para que el cliente pueda construir la cadena hasta una raíz que ya tiene instalada y en la que confía. Con www.wikipedia.org la verificación terminó con código 0, mientras que otro.ejemplo.com dio hostname mismatch y código 62. Al adelantar la fecha dos años, OpenSSL mostró certificate has expired con código 10. En mi prueba no apareció unable to get local issuer certificate, así que Kali pudo completar la cadena con su almacén de confianza.

Conclusión del experimento

Entre 2 y 4 frases con la idea principal que te llevas.

Con esta prueba vi que el certificado no solo contiene una clave pública, sino que también la relaciona con una identidad y unas fechas de validez. El cliente revisa la cadena de confianza, el nombre del servidor y la fecha. Un aviso de certificado también puede deberse a una configuración o a un reloj incorrecto, y no necesariamente a un ataque.

Preguntas de comprensión

Pregunta	Tu respuesta
¿Qué comprueba un navegador además de que la firma del certificado sea correcta?	Además de la firma del certificado, comprueba la cadena hasta una autoridad de confianza, que el nombre de la web esté incluido en el certificado y que la fecha actual esté dentro del periodo de validez.
¿Por qué el servidor debe enviar los certificados intermedios y no hace falta que envíe la raíz?	El servidor envía los intermedios para que el cliente pueda construir la cadena hasta la raíz. La raíz no hace falta enviarla porque normalmente ya está instalada en el almacén de certificados de confianza del cliente.
Un compañero ve un aviso de certificado solo en su portátil y en ninguna otra máquina. ¿Qué dos causas revisarías primero?	Primero revisaría la fecha y hora del portátil y después el almacén de certificados o la cadena intermedia. Si el aviso solo ocurre en ese equipo, esas dos causas son más probables que un problema general del servidor.

E5. Dimensionar una copia de seguridad

Pasos 5.1 a 5.5 del guión. Supuesto del guión: 1 TB de datos, 5 % de cambio diario, enlace útil de 40 MB/s, ventana de 4 horas y recuperación comprometida en 4 horas. 1 TB son 1024 x 1024 MB.

Cálculo	Tu resultado	¿Qué significa?
Total en MB (paso 5.1)	1048576 MB	Es el total de 1 TB expresado en las mismas unidades que la velocidad.
Cambio diario en MB (paso 5.1)	52428.8 MB	Es el 5 % que cambia cada día y que debe copiar la incremental.
Tiempo de la copia incremental diaria, en minutos (5.2)	21.8453 minutos	A 40 MB/s la copia diaria tarda unos 22 minutos.
¿Cabe en la ventana de 4 horas?	Sí	Cabe con mucha holgura dentro de una ventana de 4 horas.
Tiempo de la primera copia completa, en horas (5.3)	7.28178 horas	La primera copia completa tarda más de 7 horas.
¿Cabe en la ventana de 4 horas?	No	Supera la ventana nocturna de 4 horas.
MB/s necesarios para restaurar en 4 horas (5.4)	72.8178 MB/s	Es la velocidad mínima necesaria para restaurar 1 TB en 4 horas.
Gbps aproximados (5.4)	0.582542 Gbps	Equivalencia aproximada de la velocidad anterior.
Tiempo real de restauración con 40 MB/s	7.28178 horas	Con el enlace real de 40 MB/s no se cumple el objetivo de recuperación de 4 horas.

Paso 5.5, las tres preguntas de la política

Pregunta	Tu respuesta con tus datos
¿Cuánto se pierde en el peor caso?	Hasta un día de cambios, si el desastre ocurre justo antes de la siguiente copia diaria.
¿Cuánto se tarda en volver?	Unas 7.28 horas con el enlace de 40 MB/s, por lo que no se cumple el objetivo de 4 horas.
¿Cuándo se probó por última vez?	El supuesto no indica cuándo se probó por última vez. En una política real tendría que quedar registrada la fecha de la última prueba de restauración.

Qué escribí
Copia las órdenes awk de los pasos 5.1 a 5.4.

```
awk 'BEGIN { total = 1 * 1024 * 1024; cambio = total * 0.05; print "Total en MB:", total; print "Cambio diario en MB:", cambio }'
awk 'BEGIN { s = 52428.8 / 40; print "Segundos:", s; print "Minutos:", s / 60 }'
awk 'BEGIN { s = 1048576 / 40; print "Segundos:", s; print "Horas:", s / 3600 }'
awk 'BEGIN { v = 1048576 / (4 * 3600); print "MB/s necesarios:", v; print "Gbps aproximados:", v * 8 / 1000 }'
```

Lo que apareció en pantalla
Pega una captura o copia el texto de la salida de cada orden. Si una orden dio error, pégalo también.

Paso 5.1. Conversión de 1 TB a MB y cálculo del 5 % de cambio diario.

```
┌──(kali㉿kali)-[~/practica1]
└─$ awk 'BEGIN { total = 1 * 1024 * 1024; cambio = total * 0.05; print "Total en MB:", total; print "Cambio diario en MB:", cambio }'
Total en MB: 1048576
Cambio diario en MB: 52428.8

┌──(kali㉿kali)-[~/practica1]
└─$
```

Paso 5.2. Tiempo de la copia incremental diaria.

```
┌──(kali㉿kali)-[~/practica1]
└─$ awk 'BEGIN { s = 52428.8 / 40; print "Segundos:", s; print "Minutos:", s / 60 }'
Segundos: 1310.72
Minutos: 21.8453

┌──(kali㉿kali)-[~/practica1]
└─$
```

Paso 5.3. Tiempo de la primera copia completa.

```
┌──(kali㉿kali)-[~/practica1]
└─$ awk 'BEGIN { s = 1048576 / 40; print "Segundos:", s; print "Horas:", s / 3600 }'
Segundos: 26214.4
Horas: 7.28178

┌──(kali㉿kali)-[~/practica1]
└─$
```

Paso 5.4. Velocidad necesaria para cumplir una recuperación de 4 horas.

```
┌──(kali㉿kali)-[~/practica1]
└─$ awk 'BEGIN { v = 1048576 / (4 * 3600); print "MB/s necesarios:", v; print "Gbps aproximados:", v * 8 / 1000 }'
MB/s necesarios: 72.8178
Gbps aproximados: 0.582542

┌──(kali㉿kali)-[~/practica1]
└─$
```

Interpretación y análisis crítico

¿Por qué hay que pasar todo a MB antes de dividir? ¿Qué solución propones para la primera copia completa y para el tiempo de restauración? ¿Qué límite tiene este cálculo (por ejemplo, la velocidad real del enlace varía)?

Convertí todos los tamaños a MB porque la velocidad estaba en MB/s y así podía hacer los cálculos con las mismas unidades. La incremental diaria sí cabe, ya que solo mueve el 5 % de los datos y tarda unos 22 minutos, pero la primera copia completa tarda 7.28 horas y supera la ventana de 4 horas. Para esa primera carga podría sembrar el destino con un soporte físico o hacerla fuera del ciclo normal. La restauración es el punto más problemático: a 40 MB/s también tarda 7.28 horas, así que necesitaría una copia local más rápida, ampliar el enlace o revisar el tiempo comprometido. Además, el cálculo supone una velocidad sostenida constante, algo que en una red real puede variar.

Conclusión del experimento

Entre 2 y 4 frases con la idea principal que te llevas.

La incremental diaria entra de sobra en la ventana porque solo copia los cambios, pero la primera copia completa y la restauración pasan de las 4 horas. Con estos números vi que no basta con guardar los datos: también hay que comprobar cuánto tardaría realmente en recuperarlos.

Preguntas de comprensión

Pregunta	Tu respuesta
¿Por qué la copia incremental diaria cabe en la ventana y la primera copia completa no?	La incremental copia solo 52428.8 MB y tarda unos 22 minutos, mientras que la completa tiene que mover 1048576 MB y tarda unas 7.28 horas.

Pregunta	Tu respuesta
Si el tiempo de recuperación comprometido son 8 horas en lugar de 4, ¿cambia la conclusión?	Sí. Con un compromiso de 8 horas, una restauración de 7.28 horas sí entraría en el objetivo, aunque seguiría siendo recomendable dejar margen para variaciones reales de velocidad.
¿Qué haría falta cambiar, el enlace, el tiempo comprometido o la arquitectura de copias, para que la política sea viable?	Se puede ampliar el enlace, aceptar un tiempo de recuperación mayor o mejorar la arquitectura. La opción más práctica sería mantener la copia remota para desastres y disponer también de una copia local rápida para restauraciones.

Ejercicios

Para cada ejercicio entrega las órdenes que escribiste, la salida que obtuviste (texto o captura) y la respuesta razonada a las preguntas. Se valorará que expliques el porqué de cada resultado y no solo que lo obtengas. Lee todo el ejercicio antes de empezar.

Ejercicio	Apoyo	Dificultad	Tiempo
Ej1	E1	Baja	10 min.
Ej2	E1	Baja a media	10 min.
Ej3	E1 y E2	Media	10 min.
Ej4	E2	Media	15 min.
Ej5	E5	Media a alta	15 min.
Ej6	E1, E3 y E4	Alta	20 min.

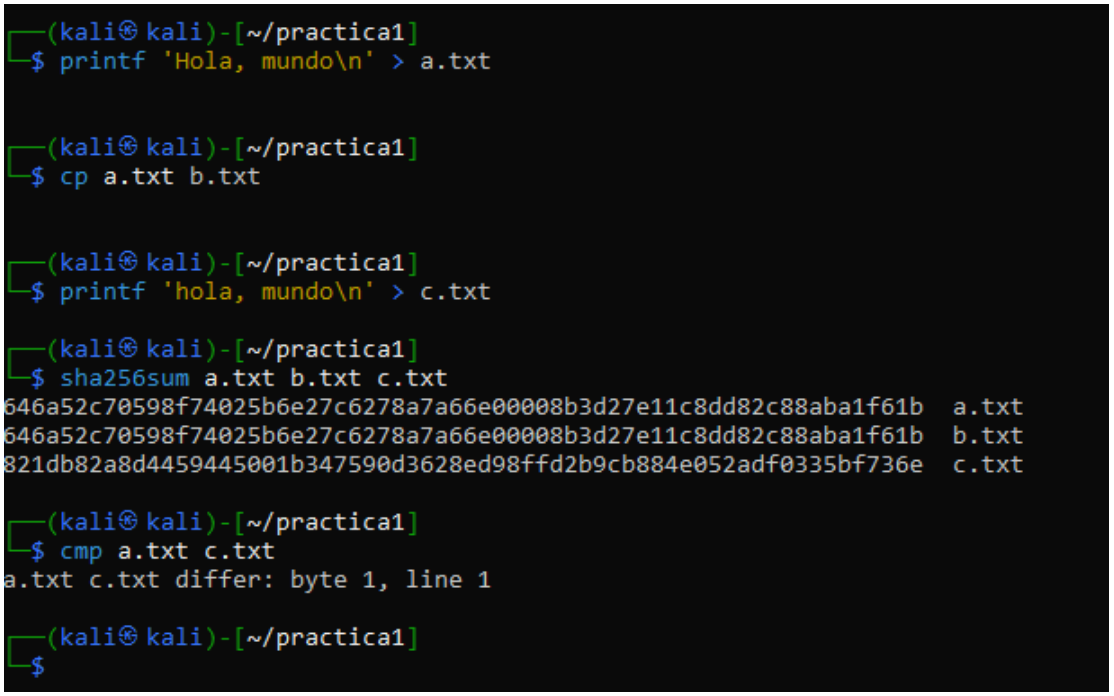
Ej1. Copiar y renombrar no cambia la huella

Órdenes que escribí
Pasos 1 y 2. Crear a.txt, copiarlo a b.txt y crear c.txt con la h en minúscula.

```
printf 'Hola, mundo\n' > a.txt
cp a.txt b.txt
printf 'hola, mundo\n' > c.txt
sha256sum a.txt b.txt c.txt
cmp a.txt c.txt
```

Salida obtenida
Huellas de los tres ficheros y salida de cmp a.txt c.txt.

Ej1. Huellas de a.txt, b.txt y c.txt y primera diferencia detectada por cmp.



Pregunta	Tu respuesta
3. ¿Qué ficheros tienen la misma huella y por qué?	a.txt y b.txt tienen la misma huella porque b.txt es una copia byte a byte de a.txt. El nombre del fichero no forma parte del contenido que calcula SHA-256.

Pregunta	Tu respuesta
4. ¿En qué byte se diferencian a.txt y c.txt? ¿Qué te dice cmp?	Se diferencian en el byte 1 de la línea 1. cmp muestra "a.txt c.txt differ: byte 1, line 1", que corresponde al cambio de H mayúscula por h minúscula.
5. ¿Cómo usarías un hash para comprobar que dos ficheros muy grandes son idénticos sin abrirlos ni compararlos byte a byte?	Calcularía SHA-256 de los dos ficheros y compararía las huellas. Si son distintas sé que los ficheros no son iguales; si coinciden, tengo una evidencia muy fuerte de que contienen los mismos datos sin tener que revisarlos manualmente.

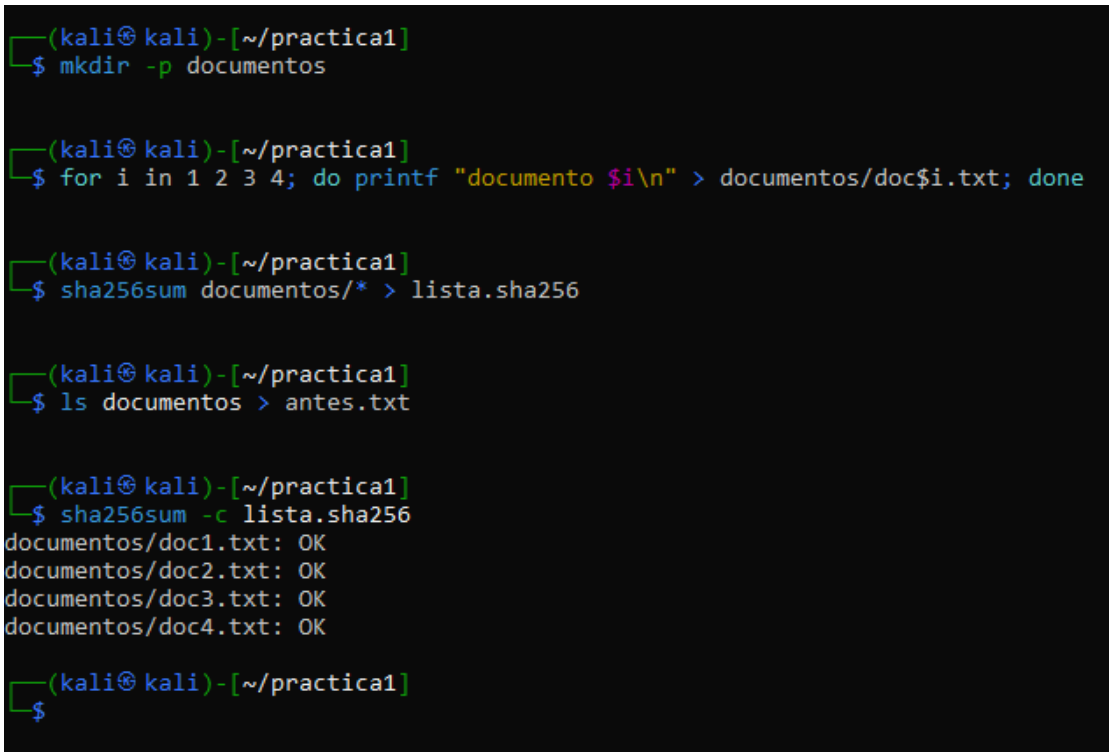
Ej2. Tres incidentes en una carpeta

Órdenes que escribí
Carpeta documentos con cuatro ficheros, manifiesto lista.sha256, antes.txt y los tres incidentes.

```
mkdir -p documentos
for i in 1 2 3 4; do printf "documento $i\n" > documentos/doc$i.txt; done
sha256sum documentos/* > lista.sha256
ls documentos > antes.txt
sha256sum -c lista.sha256
echo "modificado" >> documentos/doc2.txt
rm documentos/doc4.txt
printf "documento nuevo\n" > documentos/doc5.txt
sha256sum -c lista.sha256
ls documentos > despues.txt
diff antes.txt despues.txt
```

Salida obtenida
Comprobación del manifiesto y resultado de diff antes.txt despues.txt.

Ej2. Estado inicial: los cuatro ficheros coinciden con el manifiesto.



Ej2. Tras los incidentes: doc2 modificado, doc4 borrado y doc5 detectado por la comparación de listados.

```
┌──(kali㉿kali)-[~/practica1]
└─$ echo "modificado" >> documentos/doc2.txt

┌──(kali㉿kali)-[~/practica1]
└─$ rm documentos/doc4.txt

┌──(kali㉿kali)-[~/practica1]
└─$ printf "documento nuevo\n" > documentos/doc5.txt

┌──(kali㉿kali)-[~/practica1]
└─$ sha256sum -c lista.sha256
documentos/doc1.txt: OK
documentos/doc2.txt: FAILED
documentos/doc3.txt: OK
sha256sum: documentos/doc4.txt: No such file or directory
documentos/doc4.txt: FAILED open or read
sha256sum: WARNING: 1 listed file could not be read
sha256sum: WARNING: 1 computed checksum did NOT match

┌──(kali㉿kali)-[~/practica1]
└─$ ls documentos > despues.txt

┌──(kali㉿kali)-[~/practica1]
└─$ diff antes.txt despues.txt
4c4
< doc4.txt
---
> doc5.txt

┌──(kali㉿kali)-[~/practica1]
└─$
```

Pregunta	Tu respuesta
1. ¿Qué fichero aparece modificado y con qué mensaje? ¿Y el borrado?	doc2.txt aparece como FAILED porque su contenido cambió. doc4.txt muestra No such file or directory y FAILED open or read porque fue borrado.
2. ¿Se detecta el fichero nuevo? ¿Por qué?	No aparece en la comprobación del manifiesto porque doc5.txt no existía cuando se creó lista.sha256 y, por tanto, su nombre y su huella no estaban registrados.
3. ¿Qué información te da diff que el manifiesto no daba?	diff muestra cambios en los nombres de los ficheros: indica que doc4.txt desapareció y que apareció doc5.txt. El manifiesto por sí solo no avisaba del fichero nuevo.

Pregunta	Tu respuesta
4. Si combinas manifiesto y comparación de listados, ¿qué tipos de cambios detectas? ¿Cuál sigue sin detectarse si alguien también modifica lista.sha256?	Combinando ambos métodos detecto modificaciones de contenido, ficheros borrados y ficheros nuevos. Seguiría existiendo un problema si un atacante pudiera modificar también lista.sha256 y los listados de referencia, porque podría actualizar las referencias para ocultar el cambio.

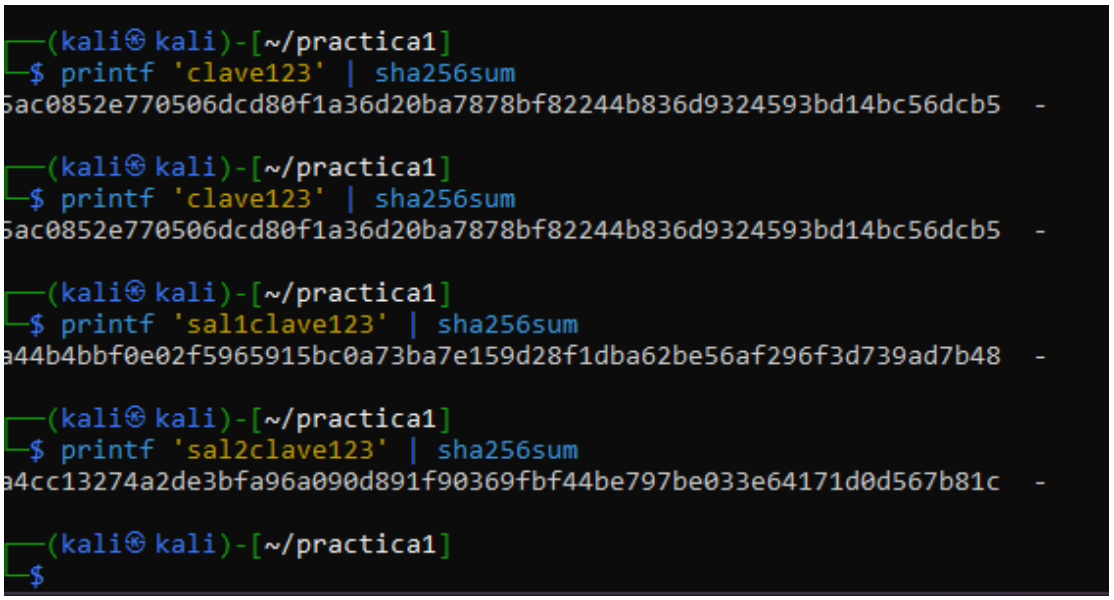
Ej3. Contraseñas, hash y sal

Órdenes que escribí
Hash de clave123 dos veces y hashes con sal1 y sal2.

```
printf 'clave123' | sha256sum
printf 'clave123' | sha256sum
printf 'sal1clave123' | sha256sum
printf 'sal2clave123' | sha256sum
```

Salida obtenida
Hashes obtenidos, con la tabla que ve el atacante.

Ej3. Hash repetido de clave123 y hashes distintos al añadir sal1 y sal2.



Pregunta	Tu respuesta
1. ¿Qué ve el atacante en la tabla? ¿Qué puede deducir de dos usuarios con el mismo hash?	El atacante ve que los dos usuarios tienen exactamente el mismo hash de clave123. Sin conocer todavía la contraseña, puede deducir que ambos han usado la misma entrada.
2. Hashes con sal para Lucía y para Marcos (escribe los primeros 8 caracteres)	Lucía: a44b4bbf. Marcos: a4cc1327.
3. ¿Por qué con sal el atacante ya no ve a simple vista que comparten contraseña?	Porque cada usuario combina la contraseña con una sal distinta. Aunque la contraseña sea la misma, la entrada que se calcula es diferente y las huellas dejan de coincidir.

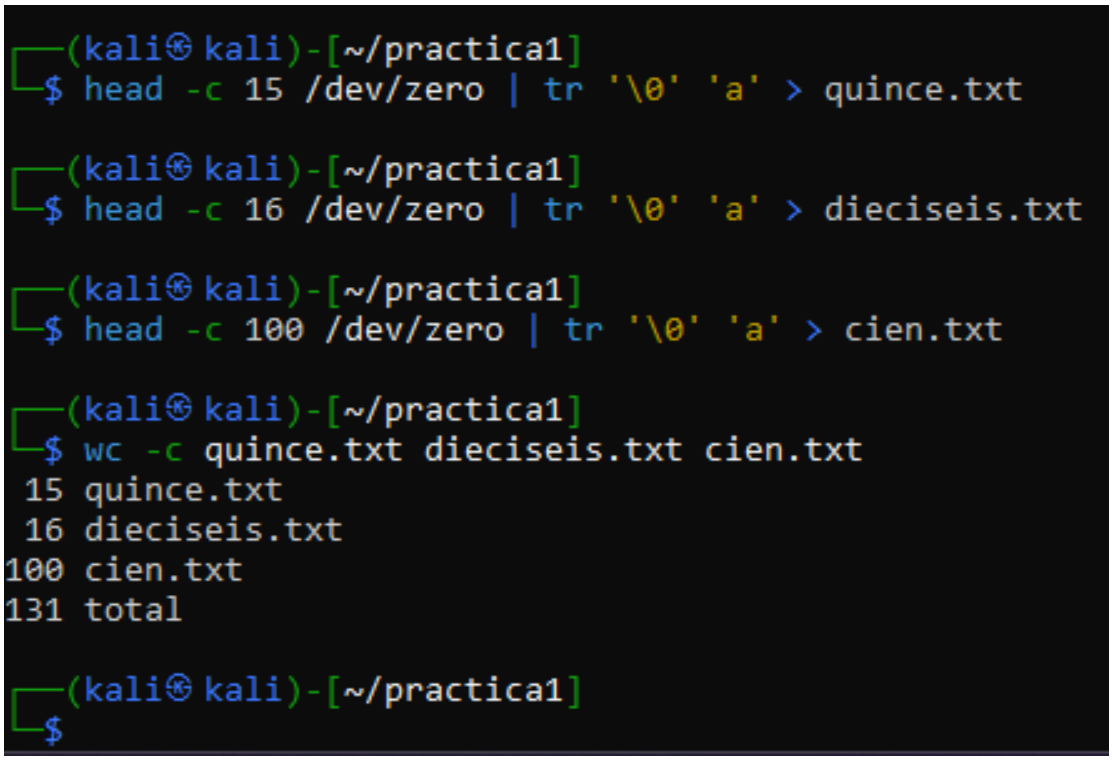
Pregunta	Tu respuesta
4. ¿Por qué la sal también impide usar una tabla de hashes de contraseñas comunes calculada de antemano?	Una tabla precalculada de hashes de contraseñas comunes ya no sirve directamente, porque el atacante tendría que volver a calcular los candidatos para cada sal concreta.
5. ¿Te conviene que la función de hash sea muy rápida o deliberadamente lenta? ¿Qué relación tiene con la opción -pbkdf2 de E2?	Conviene que la función usada para almacenar contraseñas sea deliberadamente lenta, porque así cada intento cuesta más y se reducen los intentos por segundo. PBKDF2 sigue esa idea: deriva la clave repitiendo operaciones en vez de usar directamente un hash rápido.

Ej4. Cuánto ocupa un fichero cifrado

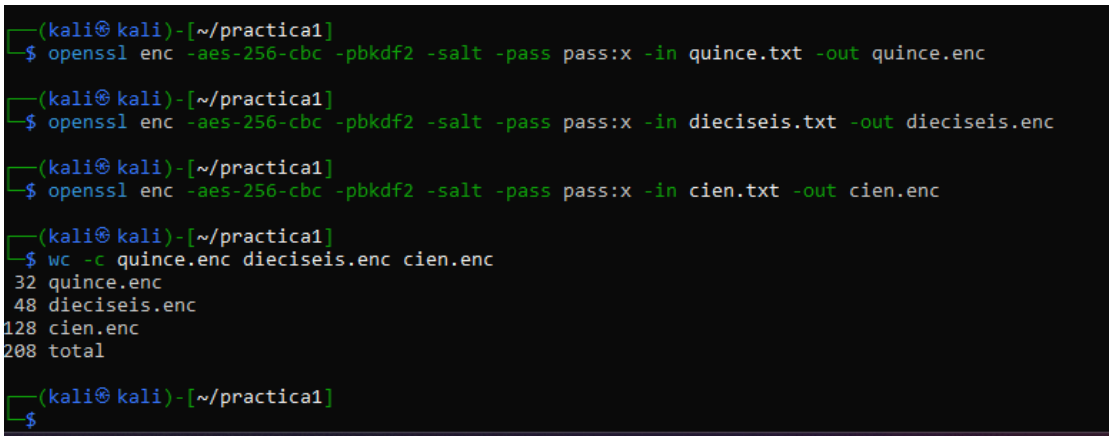
Órdenes que escribí
Crear los ficheros de 15, 16 y 100 bytes, cifrarlos con -pass pass:x y comprobar tamaños.

```
head -c 15 /dev/zero | tr '\0' 'a' > quince.txt
head -c 16 /dev/zero | tr '\0' 'a' > dieciseis.txt
head -c 100 /dev/zero | tr '\0' 'a' > cien.txt
wc -c quince.txt dieciseis.txt cien.txt
openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:x -in quince.txt -out quince.enc
openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:x -in dieciseis.txt -out dieciseis.enc
openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:x -in cien.txt -out cien.enc
wc -c quince.enc dieciseis.enc cien.enc
openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:x -in quince.txt -out quince1.enc
openssl enc -aes-256-cbc -pbkdf2 -salt -pass pass:x -in quince.txt -out quince2.enc
sha256sum quince1.enc quince2.enc
openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:x -in quince1.enc
openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:x -in quince2.enc
openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:incorrecta -in quince1.enc -out quince_mal.txt
echo $?
```

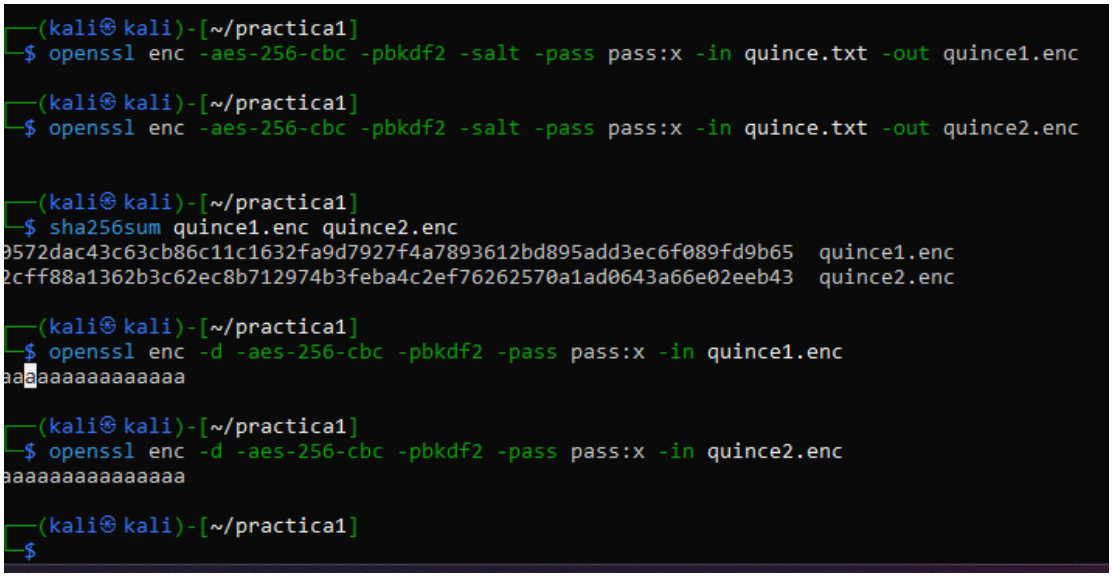
Ej4. Creación de los ficheros de 15, 16 y 100 bytes.



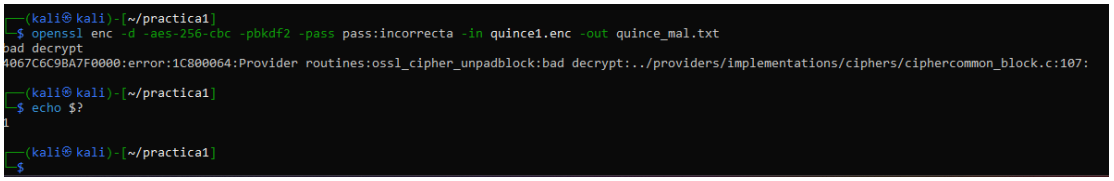
Ej4. Tamaños reales tras cifrar: 32, 48 y 128 bytes.



Ej4. Dos cifrados distintos del mismo fichero se descifran al mismo contenido.



Ej4. Contraseña incorrecta: bad decrypt y código de salida 1.



Fichero	Tamaño original	Tamaño predicho tras cifrar	Tamaño real	¿Acertaste?
quince.txt (15 bytes)	15	32	32	Sí
dieciseis.txt (16 bytes)	16	48	48	Sí
cien.txt (100 bytes)	100	128	128	Sí

Pregunta	Tu respuesta
2. Explica cómo calculaste cada predicción	Sumé la cabecera de 16 bytes y después redondeé los datos al siguiente bloque de 16 bytes. Para 15 bytes se cifra un bloque de 16; para 16 bytes se añade un bloque completo de relleno; y para 100 bytes los datos con relleno ocupan 112 bytes. Al sumar la cabecera salen 32, 48 y 128 bytes.
4. Regla general que relaciona el tamaño original con el del fichero cifrado	El tamaño cifrado es 16 bytes de cabecera más un número entero de bloques AES de 16 bytes. Siempre se añade relleno, incluso cuando el tamaño original ya es múltiplo exacto de 16.
5. Orden que demuestra que dos cifrados de quince.txt son distintos y órdenes que demuestran que los dos se descifran igual	sha256sum quince1.enc quince2.enc mostró huellas distintas. Después descifré ambos con openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:x y los dos devolvieron exactamente 15 letras a.

Pregunta	Tu respuesta
6. Si ves bad decrypt, ¿significa que nadie ha alterado el fichero? Razona con lo que viste en el paso 2.4	No. bad decrypt indica que el descifrado con esa clave terminó con un relleno no válido, pero AES-CBC por sí solo no autentica el contenido. No es una prueba de que el fichero no haya sido manipulado.

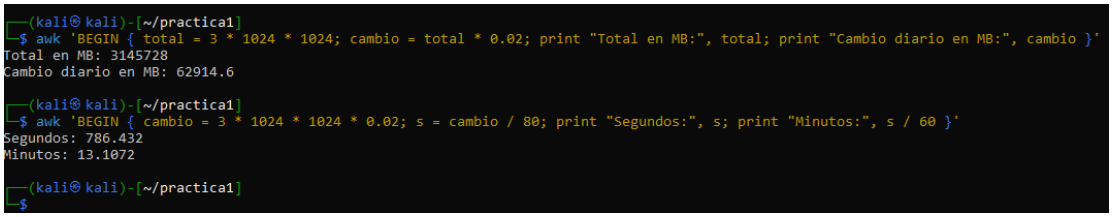
Ej5. ¿Es viable esta política de copias?

Supuesto. 3 TB de datos, un 2 % de cambio diario, un enlace útil de 80 MB/s hacia el destino remoto, una ventana nocturna de 5 horas y un tiempo de recuperación comprometido de 6 horas. 1 TB son 1024 x 1024 MB.

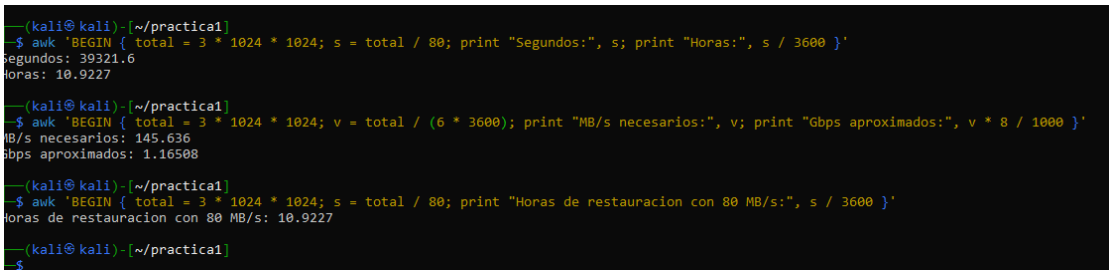
Órdenes que escribí
Las órdenes awk que usaste para cada cálculo.

```
awk 'BEGIN { total = 3 * 1024 * 1024; cambio = total * 0.02; print "Total en MB:", total; print "Cambio diario en MB:", cambio }'
awk 'BEGIN { cambio = 3 * 1024 * 1024 * 0.02; s = cambio / 80; print "Segundos:", s; print "Minutos:", s / 60 }'
awk 'BEGIN { total = 3 * 1024 * 1024; s = total / 80; print "Segundos:", s; print "Horas:", s / 3600 }'
awk 'BEGIN { total = 3 * 1024 * 1024; v = total / (6 * 3600); print "MB/s necesarios:", v; print "Gbps aproximados:", v * 8 / 1000 }'
awk 'BEGIN { total = 3 * 1024 * 1024; s = total / 80; print "Horas de restauracion con 80 MB/s:", s / 3600 }'
awk 'BEGIN { total = 3 * 1024 * 1024; s = total / 200; print "Segundos:", s; print "Horas con copia local a 200 MB/s:", s / 3600 }'
```

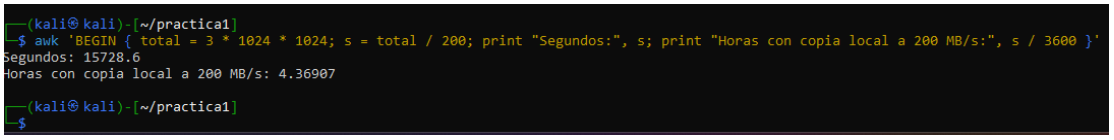
Ej5. Datos totales, cambio diario y tiempo de la copia incremental.



Ej5. Primera copia completa, velocidad necesaria para recuperar en 6 horas y tiempo con el enlace actual.



Ej5. Propuesta: restauración desde copia local a 200 MB/s en unas 4.37 horas.



Cálculo	Tu resultado	¿Cabe o se cumple?
MB totales y MB que cambian cada día	3145728 MB totales; 62914.6 MB de cambio diario	Datos base para calcular la copia.
Tiempo de la copia incremental diaria	13.1072 minutos	Sí. Cabe de sobra en la ventana de 5 horas.
Tiempo de la primera copia completa	10.9227 horas	No. Supera la ventana de 5 horas.

Cálculo	Tu resultado	¿Cabe o se cumple?
Velocidad necesaria para restaurar todo en 6 horas	145.636 MB/s (aprox. 1.16508 Gbps)	Es la velocidad sostenida mínima para restaurar 3 TB en 6 horas.
Tiempo de restauración con el enlace actual	10.9227 horas	No se cumple el compromiso de 6 horas con el enlace de 80 MB/s.
Tiempo de restauración con tu propuesta	4.36907 horas con una copia local a 200 MB/s	Sí. La propuesta permite recuperar dentro de las 6 horas.

Pregunta	Tu respuesta
2. Si la primera copia completa no cabe en la ventana, ¿qué harías?	Haría la primera copia completa fuera de la ventana habitual, por ejemplo sembrando el destino con un soporte físico o realizando la carga inicial en un periodo planificado más largo. Después las incrementales diarias sí caben en la ventana.
4. Tu propuesta de cambio concreto en la arquitectura de copias, demostrada con números (incluye la velocidad del soporte local que elijas)	Mantendría la copia remota de 80 MB/s para protegerme ante un desastre del edificio y añadiría una copia local rápida para restauraciones. Con una velocidad local sostenida de 200 MB/s, restaurar 3 TB tarda 4.36907 horas, por debajo del compromiso de 6 horas.
5. Las tres preguntas de la política. ¿Cuánto se pierde en el peor caso? ¿Cuánto se tarda en volver? ¿Cuándo se probó por última vez?	Pérdida máxima: hasta un día de cambios si la copia es diaria. Tiempo para volver: unas 4.37 horas usando la copia local propuesta; con el enlace remoto actual serían 10.92 horas. Última prueba: el supuesto no da ninguna fecha, por lo que una política real debería registrar cuándo se comprobó por última vez la restauración.

Ej6. Ana, Beto y Eve

Órdenes que escribí
Copias entre carpetas, verificación de Beto, ataque de Eve con su propio par de claves y comparación de huellas.

```
mkdir -p ana beto eve
cd ana
openssl genpkey -algorithm RSA -out privada.pem -pkeyopt rsa_keygen_bits:3072
openssl pkey -in privada.pem -pubout -out publica.pem
printf 'Contrato: Ana vende a Beto 10 equipos por 5000 EUR\n' > contrato.txt
openssl dgst -sha256 -sign privada.pem -out contrato.sig contrato.txt
cd ..
cp ana/contrato.txt ana/contrato.sig ana/publica.pem beto/
openssl dgst -sha256 -verify beto/publica.pem -signature beto/contrato.sig beto/contrato.txt
cp ana/contrato.txt ana/contrato.sig ana/publica.pem eve/
printf 'Contrato: Ana vende a Beto 10 equipos por 9000 EUR\n' > eve/contrato.txt
cp eve/contrato.txt beto/contrato.txt
cp eve/contrato.sig beto/contrato.sig
openssl dgst -sha256 -verify beto/publica.pem -signature beto/contrato.sig beto/contrato.txt
openssl genpkey -algorithm RSA -out eve/privada_eve.pem -pkeyopt rsa_keygen_bits:3072
openssl pkey -in eve/privada_eve.pem -pubout -out eve/publica_eve.pem
openssl dgst -sha256 -sign eve/privada_eve.pem -out eve/contrato_eve.sig eve/contrato.txt
cp eve/contrato.txt beto/contrato.txt
cp eve/contrato_eve.sig beto/contrato.sig
cp eve/publica_eve.pem beto/publica.pem
openssl dgst -sha256 -verify beto/publica.pem -signature beto/contrato.sig beto/contrato.txt
sha256sum ana/publica.pem eve/publica_eve.pem
```

Salida obtenida
Resultados de cada verificación y las dos huellas de clave pública del paso 5.

Ej6, paso 1. Beto verifica correctamente el contrato original de Ana.

```
┌──(kali㉿kali)-[~/practica1]
└─$ cp ana/contrato.txt ana/contrato.sig ana/publica.pem beto/

┌──(kali㉿kali)-[~/practica1]
└─$ openssl dgst -sha256 -verify beto/publica.pem -signature beto/contrato.sig beto/contrato.txt
Verified OK

┌──(kali㉿kali)-[~/practica1]
└─$
```

Ej6, paso 2. El contrato alterado a 9000 EUR no verifica con la firma original de Ana.

```
┌──(kali㉿kali)-[~/practica1]
└─$ cp ana/contrato.txt ana/contrato.sig ana/publica.pem eve/

┌──(kali㉿kali)-[~/practica1]
└─$ printf 'Contrato: Ana vende a Beto 10 equipos por 9000 EUR\n' > eve/contrato.txt

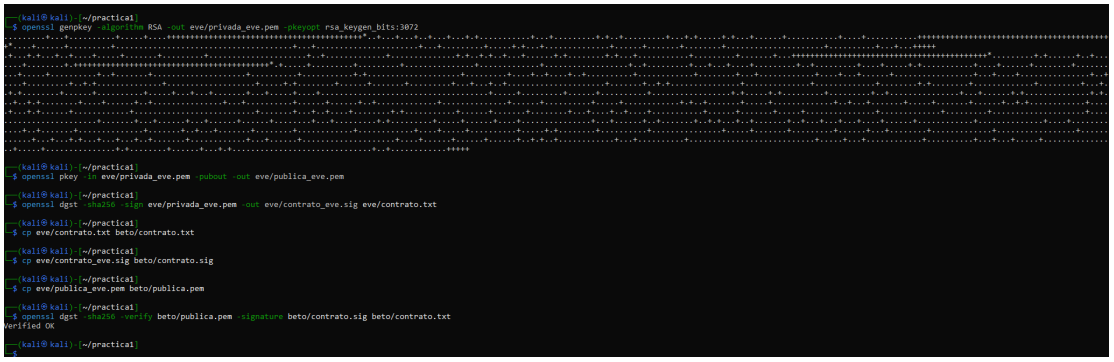
┌──(kali㉿kali)-[~/practica1]
└─$ cp eve/contrato.txt beto/contrato.txt

┌──(kali㉿kali)-[~/practica1]
└─$ cp eve/contrato.sig beto/contrato.sig

┌──(kali㉿kali)-[~/practica1]
└─$ openssl dgst -sha256 -verify beto/publica.pem -signature beto/contrato.sig beto/contrato.txt
Verification failure
4067EAD44F7F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:441:
4067EAD44F7F0000:error:1C880004:Provider routines:rsa_verify_directly:RSA lib:providers/implementations/signature/rsa_sig.c:1038:

┌──(kali㉿kali)-[~/practica1]
└─$
```

Ej6, paso 3. Eve firma el contrato alterado con su propia clave y, al entregar también su pública, Beto obtiene Verified OK.



Ej6, paso 5. Las huellas de las claves públicas de Ana y Eve son diferentes.

```
┌──(kali㉿kali)-[~/practica1]
└─$ sha256sum ana/publica.pem eve/publica_eve.pem
7f6aa86b711215c94211a9128d9a85d708a8800e18739d18573dd8165801d608  ana/publica.pem
c13675543bf1fb5e328804d4dc98a68b3167f45ccb6b5f3f6440bcee09394fa4  eve/publica_eve.pem

┌──(kali㉿kali)-[~/practica1]
└─$
```

Paso	Resultado que obtuvo Beto
1. Verificación de la firma de Ana, sin cambios	Verified OK
2. Verificación con el importe cambiado y la firma reutilizada	Verification failure (bad signature)
3. Verificación con la firma y la clave pública de Eve	Verified OK
5. Huella (primeros 8 caracteres) de la clave de Ana y de la de Eve	Ana: 7f6aa86b · Eve: c1367554

Pregunta	Tu respuesta
2. ¿Verifica Beto la firma de Eve reutilizada? ¿Por qué?	No. Beto obtiene Verification failure porque Eve cambió el importe pero reutilizó la firma creada por Ana para el contrato original de 5000 EUR. La firma está vinculada al contenido exacto.
3. ¿Qué obtiene Beto en el paso 3?	Obtiene Verified OK, porque el contrato de 9000 EUR sí está firmado con la clave privada de Eve y Beto está verificándolo con la clave pública de Eve que ella le presentó como si fuera la de Ana.
4. ¿Qué ha fallado? ¿Se ha roto la criptografía? ¿Qué no tenía forma de comprobar Beto?	No se ha roto la criptografía. La firma y la verificación funcionan correctamente; lo que falla es la autenticidad de la clave pública. Beto no tenía una forma independiente de comprobar que la clave que había recibido pertenecía realmente a Ana.
5. ¿Cómo descubre Beto el engaño con las huellas?	Ana le comunica la huella de su clave pública por otro canal. La huella real empieza por 7f6aa86b y la clave entregada por Eve por c1367554, así que Beto ve que no son la misma clave.
6. ¿Cómo lo resolvería un certificado? ¿Quién firmaría la clave pública de Ana y qué comprobaría Beto antes de fiarse?	Una autoridad de certificación de confianza firmaría un certificado que vinculara la identidad de Ana con su clave pública. Beto comprobaría la firma de la autoridad, la cadena de confianza, la identidad y la vigencia antes de aceptar esa clave.
7. Beto quiere responder a Ana con un secreto. ¿Con qué clave lo cifra? ¿Qué ocurriría si hubiera usado la clave de Eve?	Beto debe cifrar el secreto con la clave pública real de Ana, para que solo ella pueda descifrarlo con su privada. Si usara la clave pública de Eve, Eve podría leer el secreto y Ana no podría descifrarlo con su clave privada.

Desafíos (opcionales)

Son opcionales, se hacen fuera de clase y suman punto extra que el profesor añade aparte de la rúbrica. Si haces alguno, marca aquí cuáles y entrega en cada uno las órdenes o el código, la salida y las respuestas. Si adjuntas un script como archivo aparte, escribe su nombre.

Desafío	Apoyo	Dificultad	Tiempo	¿Lo he hecho? (Sí/No)
D1	E4	Media	30 min.	No
D2	E1 y E3	Media a alta	45 min.	No
D3	E2 y E3	Alta	45 min.	No
D4	E3 y E4	Muy alta	60 min.	No

D1. Radiografía de tres certificados y diagnóstico de avisos

Web	Titular	Emisor	Válido desde	Válido hasta	Nº de certificados

Pregunta	Tu respuesta
2. ¿Qué observas al comparar emisores y vigencias? ¿Coinciden los emisores? ¿Son parecidas las vigencias?	
3. ¿Caduca alguno en los próximos 30 días? Escribe la orden y el resultado	

Causa habitual de aviso	Orden para diagnosticarla	Salida que buscarías
Caducado		
El nombre no coincide		
Falta el intermedio		
El reloj del cliente está mal		

D2. Vigilante de integridad

Script (pégalo aquí o indica el nombre del archivo adjunto)
Debe tener los modos crear y comprobar, incluir las subcarpetas e informar de ficheros modificados, borrados y nuevos.

Salida de la prueba
Carpeta con cinco ficheros y una subcarpeta, provocando los tres tipos de cambios.

Ampliación opcional, manifiesto firmado
Explica qué ataque impide que el script verifique la firma antes de comparar y cuál no impide.

D3. Cifrado híbrido

Órdenes de Ana y de Beto, paso a paso
Escribe quién tiene qué clave en cada paso y las órdenes que ejecuta.

Salida y comprobación con cmp
Debe verse que el fichero descifrado es idéntico al original.

Pregunta	Tu respuesta
5. ¿Por qué no se cifra el fichero de 1 MB directamente con RSA y por qué no se envía la clave de AES en claro?	
6. ¿Qué parte del proceso protege la confidencialidad y qué ocurre con la integridad? Si un atacante altera el fichero cifrado, ¿lo detectaría Beto?	
7. Ampliación opcional. ¿Firmarías el original y cifrarías después, o cifrarías y firmarías el resultado? Justifica	

D4. Una mini PKI

Órdenes que escribí *Pasos 1 a 4, la autoridad Mi CA de prueba, la solicitud para servidor.local, la firma con subjectAltName y openssl verify.*

Salida obtenida *Titular, emisor, vigencia y nombres del certificado, y el resultado de la verificación.*

Fallo provocado	Orden usada	Código de error	Causa de aviso habitual que ilustra
Una autoridad distinta no valida el certificado			
Otro nombre (-verify_hostname)			
Dentro de dos años (-attime)			

7. ¿Qué debería instalar un cliente para confiar en los certificados de tu autoridad y qué riesgo supone instalar una raíz en los equipos de otras personas?

Reflexión final y lista de comprobación

Cierra la práctica relacionando cada técnica con el principio de la tríada CIA que protege.

Técnica	Principio CIA que protege	Un ejemplo de lo que hiciste en la práctica
Hash	Integridad	Calculé y comparé huellas SHA-256 para detectar cambios en ficheros y manifiestos.
Cifrado simétrico	Confidencialidad	Cifré mensaje.txt con AES-256-CBC y recuperé el contenido con la misma contraseña.
Firma digital	Integridad	Firmé mensaje.txt con la clave privada y comprobé que un cambio provocaba Verification failure.
Cifrado asimétrico	Confidencialidad	Cifré secreto.txt con la clave pública y lo descifré únicamente con la clave privada correspondiente.
Certificado y PKI	Integridad / autenticidad	Verifiqué el certificado de Wikipedia, el nombre del servidor, la cadena y las fechas de validez.
Copias 3-2-1	Disponibilidad	Calculé si una política de copias permitía copiar y restaurar los datos dentro de los tiempos comprometidos.

1. La idea más importante que me llevo de la práctica, y por qué

Me quedo sobre todo con que cada técnica cubre una necesidad distinta. El hash sirve para detectar cambios, el cifrado para proteger la confidencialidad, la firma para comprobar integridad y origen, y el certificado para relacionar una clave pública con una identidad. Haciendo las pruebas vi que ninguna de ellas sustituye a las demás.

2. Un error o duda que tuve y cómo lo resolví

Al retomar la práctica, la máquina virtual Ubuntu con la que había realizado Paso 0, E1 y E2 dejó de arrancar correctamente y perdí el entorno de trabajo. Preparé una nueva máquina Kali, reconstruí los ficheros necesarios y continué desde E3. Durante E3 también tuve varios errores de sintaxis, como escribir -pkeyout en lugar de -pkeyopt u omitir -sign/-encrypt, que corregí revisando el comando del guion antes de repetirlo.

3. Una situación real donde usaría lo que he aprendido
Por ejemplo, revisar un aviso de certificado o comprobar una descarga.

Por ejemplo, si tuviera que descargar o enviar un fichero importante, usaría el hash para comprobar que no ha cambiado, lo cifraría si contiene información sensible y verificaría la firma si estuviera disponible. También revisaría el certificado HTTPS antes de asumir que un aviso es un ataque. En una política de copias, usaría los cálculos de E5 para comprobar si realmente puedo recuperar los datos dentro del tiempo previsto.

Entrega final, el archivo zip

Además de este cuaderno, debes entregar un archivo zip con lo que has generado en la práctica. Sirve para que el profesor pueda volver a comprobar tus resultados, por ejemplo verificar tus firmas o tu manifiesto, con los ficheros reales de tu máquina. Crea el zip al final, cuando hayas terminado todos los ejercicios, y adjunta en la entrega los dos archivos, este cuaderno y el zip.

Qué entra en el zip

Contenido	De dónde sale	¿Se incluye?
Toda la carpeta practica1 (ficheros de texto, manifiestos y listados)	Paso 0 y experimentos E1 a E5	Sí
Ficheros cifrados, claves y firmas de prueba (.enc, .pem, .sig)	E2 y E3	Sí, son claves de prueba
Carpetas documentos, ana, beto y eve, con sus ficheros	Ej2 y Ej6	Sí
Scripts y ficheros de los desafíos que hayas hecho	D1 a D4	Sí, solo los que hayas hecho
grande.bin (10 MiB) y largo.txt	Paso 1.4 y paso 3.6	No, ocupan sitio y no aportan
Cualquier clave o contraseña real que uses en otro sitio	No pertenece a esta práctica	Nunca

Cómo crear el zip

Tu sistema	Pasos
Windows (Git Bash)	Primero borra grande.bin o muévelo fuera de la carpeta. Después abre el Explorador de archivos, entra en tu carpeta personal y localiza la carpeta practica1. Con el botón derecho elige Comprimir en archivo ZIP, o Enviar a y Carpeta comprimida. Cambia el nombre del zip a Practica1_SAD_Apellido_Nombre.zip. Git Bash no incluye la orden zip, por eso se hace desde el Explorador. O directamente si ssh está instalado en el kali y accedes desde el powershell y te pasas el archivo por ahí, tambien funciona perfectamente que es como lo he hecho yo.
Linux o macOS	En la terminal escribe cd ~ y después zip -r Practica1_SAD_Apellido_Nombre.zip practica1 -x "practica1/grande.bin". Si no tienes la orden zip, instálala con tu gestor de paquetes.

Comprueba que el zip está bien

Antes de entregar, mira el contenido del zip sin abrirlo. En Linux o macOS escribe unzip -l Practica1_SAD_Apellido_Nombre.zip. En Windows ábrelo con doble clic y revisa la lista. Debe aparecer la carpeta practica1 con sus ficheros y no debe aparecer grande.bin.

Dato	Tu respuesta
Nombre exacto del zip que entrego	Practica1_SAD_Hosannah_Pedro.zip
Tamaño del zip	24 KB
¿Incluye los desafíos? ¿Cuáles?	No. No he realizado los desafíos opcionales.

Contenido de mi zip
Pega aquí el listado que muestra unzip -l, o una captura del zip abierto con todos los ficheros a la vista.

El ZIP contiene 59 entradas de la carpeta practica1. unzip -t terminó con “No errors detected in compressed data” y la búsqueda de grande.bin y largo.txt no devolvió resultados, así que los dos quedaron fuera del ZIP.

Listado del contenido del ZIP con unzip -l: primera parte.

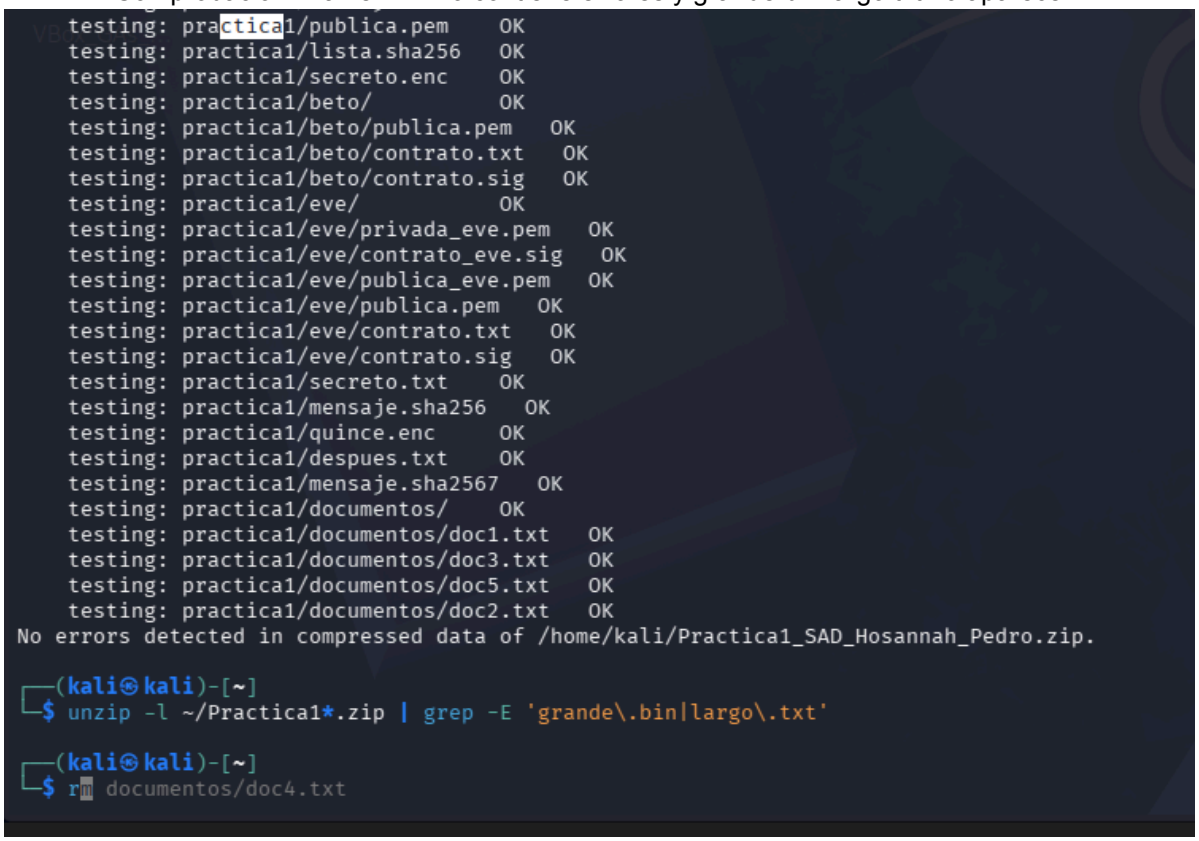
```
┌──(kali㉿kali)-[~]
└─$ unzip -l ~/Practica1*.zip
Archive:  /home/kali/Practica1_SAD_Hosannah_Pedro.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-10-14 12:01   practica1/
       36  2026-10-14 11:44   practica1/antes.txt
       48  2026-10-14 11:52   practica1/dieciseis.enc
        0  2026-10-14 11:56   practica1/quince_mal.txt
      128  2026-10-14 11:52   practica1/cien.enc
        0  2026-10-14 12:01   practica1/ana/
     2484  2026-10-14 12:01   practica1/ana/privada.pem
      625  2026-10-14 12:01   practica1/ana/publica.pem
       51  2026-10-14 12:01   practica1/ana/contrato.txt
      384  2026-10-14 12:01   practica1/ana/contrato.sig
       12  2026-10-14 11:43   practica1/c.txt
       15  2026-10-14 11:50   practica1/quince.txt
       32  2026-10-14 11:53   practica1/quince1.enc
      384  2026-10-14 10:35   practica1/grande.sig
       16  2026-10-14 11:51   practica1/dieciseis.txt
        0  2026-10-14 11:08   practica1/largo.enc
       12  2026-10-14 11:43   practica1/a.txt
       30  2026-10-14 08:44   practica1/mensaje2.txt
       12  2026-10-14 11:43   practica1/b.txt
      384  2026-10-14 10:34   practica1/mensaje.sig
       32  2026-10-14 11:53   practica1/quince2.enc
        0  2026-10-14 08:45   practica1/vacio.txt
      395  2026-10-14 08:56   practica1/manifiesto.sha256
       29  2026-10-14 10:39   practica1/mensaje.txt
      100  2026-10-14 11:51   practica1/cien.txt
     1700  2026-10-14 11:07   practica1/privada2.pem
       30  2026-10-14 08:44   practica1/mensaje3.txt
     2484  2026-10-14 10:27   practica1/privada.pem
        0  2026-10-14 08:58   practica1/copia/
       24  2026-10-14 08:56   practica1/copia/f1.txt
        6  2026-10-14 08:58   practica1/copia/nuevo.txt
       30  2026-10-14 08:57   practica1/copia/f2.txt
       24  2026-10-14 08:56   practica1/copia/f3.txt
       24  2026-10-14 08:56   practica1/copia/f5.txt
       30  2026-10-14 08:45   practica1/mensaje4.txt
      625  2026-10-14 10:27   practica1/publica.pem
      344  2026-10-14 11:44   practica1/lista.sha256
      384  2026-10-14 11:06   practica1/secreto.enc
        0  2026-10-14 12:02   practica1/beto/
      625  2026-10-14 12:05   practica1/beto/publica.pem
       51  2026-10-14 12:05   practica1/beto/contrato.txt
      384  2026-10-14 12:05   practica1/beto/contrato.sig
        0  2026-10-14 12:05   practica1/eve/
     2484  2026-10-14 12:05   practica1/eve/privada_eve.pem
      384  2026-10-14 12:05   practica1/eve/contrato_eve.sig
      625  2026-10-14 12:05   practica1/eve/publica_eve.pem
```

Listado del contenido y comienzo de la comprobación de integridad con unzip -t.

```
      384  2026-10-14 12:03   practica1/eve/contrato.sig
       30  2026-10-14 11:06   practica1/secreto.txt
       78  2026-10-14 08:46   practica1/mensaje.sha256
       32  2026-10-14 11:52   practica1/quince.enc
       36  2026-10-14 11:46   practica1/despues.txt
       78  2026-10-14 08:46   practica1/mensaje.sha2567
        0  2026-10-14 11:46   practica1/documentos/
       12  2026-10-14 11:44   practica1/documentos/doc1.txt
       12  2026-10-14 11:44   practica1/documentos/doc3.txt
       16  2026-10-14 11:46   practica1/documentos/doc5.txt
       23  2026-10-14 11:46   practica1/documentos/doc2.txt
---------                     -------
    16844                     59 files

┌──(kali㉿kali)-[~]
└─$ unzip -t ~/Practica1*.zip
Archive:  /home/kali/Practica1_SAD_Hosannah_Pedro.zip
    testing: practica1/                   OK
    testing: practica1/antes.txt          OK
    testing: practica1/dieciseis.enc      OK
    testing: practica1/quince_mal.txt     OK
    testing: practica1/cien.enc           OK
    testing: practica1/ana/               OK
    testing: practica1/ana/privada.pem    OK
    testing: practica1/ana/publica.pem    OK
    testing: practica1/ana/contrato.txt   OK
    testing: practica1/ana/contrato.sig   OK
    testing: practica1/c.txt              OK
    testing: practica1/quince.txt         OK
    testing: practica1/quince1.enc        OK
    testing: practica1/grande.sig         OK
    testing: practica1/dieciseis.txt      OK
    testing: practica1/largo.enc          OK
    testing: practica1/a.txt              OK
    testing: practica1/mensaje2.txt       OK
    testing: practica1/b.txt              OK
    testing: practica1/mensaje.sig        OK
    testing: practica1/quince2.enc        OK
    testing: practica1/vacio.txt          OK
    testing: practica1/manifiesto.sha256  OK
    testing: practica1/mensaje.txt        OK
    testing: practica1/cien.txt           OK
    testing: practica1/privada2.pem       OK
    testing: practica1/mensaje3.txt       OK
    testing: practica1/privada.pem        OK
    testing: practica1/copia/             OK
    testing: practica1/copia/f1.txt       OK
    testing: practica1/copia/nuevo.txt    OK
    testing: practica1/copia/f2.txt       OK
    testing: practica1/copia/f3.txt       OK
    testing: practica1/copia/f5.txt       OK
    testing: practica1/mensaje4.txt       OK
    testing: practica1/publica.pem        OK
```

Comprobación final: el ZIP no contiene errores y grande.bin/largo.txt no aparecen.



Lista de comprobación antes de entregar

Marca si está hecho	✔
He guardado el archivo como Practica1_SAD_Apellido_Nombre	✓
He rellenado los datos del alumno y la versión de OpenSSL	✓
Cada ejercicio lleva las órdenes, la salida (texto o captura) y una explicación razonada	✓
He escrito mis predicciones antes de ejecutar y he anotado si acerté	✓
En E4 he indicado la fecha en que hice la prueba	✓
Ninguna clave privada ni contraseña real aparece en el cuaderno	✓
Las explicaciones están escritas con mis palabras y no copiadas de la guía	✓
Si he hecho desafíos, he indicado cuáles en la tabla de desafíos	✓
He creado el zip con la carpeta practica1, sin grande.bin, y he comprobado su contenido	✓
He guardado el zip como Practica1_SAD_Apellido_Nombre.zip	✓
Adjunto en la entrega los dos archivos, el cuaderno y el zip	Pendiente de adjuntar ambos archivos en la plataforma de entrega